TeleBook Backend — API Reference

Complete reference for the TeleBook messaging backend: every HTTP endpoint, every real-time event, all data models and the full feature list.

Base URL: `https://api.telebookservices.com`
Backend revision: branch `new-fix-14sep26`, 18 September 2026
Endpoints: 112 HTTP routes + Socket.IO events

> **How the examples were produced.** Every request and response in this document was captured by calling a running copy of this backend on 18 September 2026 — 123 successful calls and every documented error case — not written by hand. Field names, status codes and response shapes are therefore exactly what the server returns. Ids, names and timestamps are from test data. Long arrays are cut to two items (`"…N more"`) and JWTs are shortened.

Contents

1. Read this first
2. Basics
3. Feature list
4. Common flows
5. Endpoint index
6. Endpoints
7. Real-time (Socket.IO)
8. Push notifications (FCM)
9. Data models
10. Enumerations
11. Server configuration
12. Known limitations
13. Verification checklist

1. Read this first

These are the rules most often gotten wrong. An AI assistant integrating the app should apply all of them before anything else.

1. **Every private request needs `Authorization: Bearer <token>`.** The token comes from `POST /api/auth/verify-otp`.
2. **New users must complete their profile first.** Endpoints marked *Token + complete profile* return `403` with `isProfileComplete: false` and `redirectTo: "/complete-profile"` until `POST /api/auth/complete-profile` succeeds. Handle this 403 globally by routing to that screen.
3. **Every response has `success: true | false`.** Check it, not only the HTTP status.
4. **Stored files are keys, not URLs.** `profilePicture`, `groupIcon`, `channelIcon`, `broadcastIcon`, `attachments[].url` and `mediaUrl` hold keys like `uploads/images/…jpg`. Display them as `https://api.telebookservices.com/media/<key>`, which redirects to a signed URL. Empty string means no file.
5. **Send messages over HTTP only** (`POST /api/messages`). The server delivers them over the socket and by push. Never emit the socket event `new-message`.
6. **To attach a file:** upload it with `POST /api/users/media` (multipart field `media`), then send the returned `data` object inside `attachments` of `POST /api/messages`.
7. **Field formats differ per endpoint — follow each table exactly:**
   - `users` (create group) and `recipients` (broadcasts): an array of ids, or `JSON.stringify` of it when sending FormData.
   - `userIds` (add to group) and `messageIds` / `chatIds`: always a real JSON array.
   - `data` (contact cache): always a **string** — `JSON.stringify` your object.
   - `duration` (mute): a **number** of hours, `0` = forever.
8. **`GET /api/messages/:chatId` returns newest first.** Reverse for display. Calling it also marks the chat read — only call it when the chat is actually opened.
9. **Pagination totals have endpoint-specific names** (`totalChats`, `totalGroups`, `totalMessages` …). Use `pagination.hasMore` to decide whether to load more.
10. **Endpoints with a file upload accept `multipart/form-data`.** When not sending a file, a plain JSON body works too.
11. **Push notifications need the token re-sent on every app start** (`POST /api/notifications/save-token`), not only at login — see section 8. On logout call `POST /api/auth/logout`.
12. **403 `You are not part of this chat` means the chat id is not one of the user's chats.** Only use chat ids from `GET /api/chats`. After leaving a group, drop it from local state.
13. **Deleted accounts stay in chats as `"Deleted account"`** with empty photo and about. Sending to one returns 403. Never assume `users` has another real person.
14. **Edited messages** carry `isEdited: true` and `editedAt`; show an "edited" label.
15. **Some behaviour depends on server flags** — see Server configuration and Known limitations before reporting a bug.

2. Basics

Access levels

ACCESS	REQUIREMENT	FAILURE
Public	none	—
Token	valid JWT in `Authorization: Bearer <token>`	`401` `{"success":false,"message":"Not authorized – no token provided"}` (or `invalid or expired token`, `User no longer exists`, `Account not verified…`)
Token + complete profile	as above, and the user has a `name`	`403` `{"success":false,"isProfileComplete":false,"message":"Please complete your profile first…","redirectTo":"/complete-profile"}`

Response envelope

```
// success
{ "success": true, "...": "endpoint-specific fields" }

// error
{ "success": false, "message": "Human-readable reason", "error": "technical detail (500s only)" }
```

- `400` invalid input · `401` not logged in · `403` not allowed · `404` not found · `429` rate limited · `500` server error.
- Rate limit: 1000 requests per minute per IP → `429 {"success":false,"message":"Too many requests. Please slow down."}`.
- Ids are 24-character hex strings (MongoDB ObjectId). Dates are ISO 8601 strings in UTC.
- Request bodies up to 50 MB (`application/json` or `multipart/form-data`). Single uploaded files up to 100 MB.

Pagination

List endpoints accept `page` (default 1) and `limit` (default 50 unless noted) and return:

```
"pagination": { "currentPage": 1, "totalPages": 3, "totalChats": 120, "hasMore": true, "limit": 50 }
```

ENDPOINT	TOTAL FIELD
`GET /api/chats`	`totalChats`
`GET /api/chats/groups`	`totalGroups`
`GET /api/messages/:chatId`	`totalMessages`
`GET /api/users/contacts`	`totalContacts`
`GET /api/channels`	`totalChannels`
`GET /api/channels/:channelId/messages`	`totalMessages`
`GET /api/broadcasts`	`totalBroadcasts`
`GET /api/broadcasts/:broadcastId/messages`	`totalMessages`

File uploads

ENDPOINT	MULTIPART FIELD	FILES
`PUT /api/users/profile`	`profilePicture`	image
`POST /api/users/media`	`media`	any file
`POST /api/chats/group`	`groupIcon`	image
`PUT /api/chats/group/update`	`groupIcon`	image
`POST /api/status`	`media`	image or video
`POST /api/channels`	`channelIcon`	image
`PUT /api/channels/:channelId/update`	`channelIcon`	image
`POST /api/channels/:channelId/messages`	`files`	up to 5 files
`POST /api/broadcasts`	`broadcastIcon`	image
`PUT /api/broadcasts/:broadcastId`	`broadcastIcon`	image
`POST /api/broadcasts/:broadcastId/messages`	`files`	up to 5 files

React Native `FormData` example:

```
const form = new FormData();
form.append('media', { uri: file.uri, name: file.fileName ?? 'photo.jpg', type: file.mimeType ?? 'image/jpeg' });
const res = await fetch('https://api.telebookservices.com/api/users/media', {
  method: 'POST',
  headers: { Authorization: `Bearer ${token}` },   // do NOT set Content-Type — fetch adds the boundary
  body: form,
});
const { data } = await res.json();   // { url, filename, fileType, fileSize }
```

3. Feature list

Everything the backend supports. See Known limitations for what is partial.

Account & login

- Phone number + OTP login, no password
- New vs returning user detection (`isNewUser`)
- Resend OTP
- One-time profile completion (name, city)
- JWT sessions valid 30 days
- Session restore via `GET /api/auth/me`
- Push token registration at login and on token rotation
- Account deletion: data removed, account shown to others as "Deleted account", number can register again

Profile

- Edit name, about (139 chars), district, city, state
- Profile picture upload (old picture deleted automatically)
- Personal invite code and invite lookup
- View another user's public profile

Contacts

- Phone-book sync: find which contacts use the app
- In-app contact book: add by mobile, custom display names, remove, paginated list with presence
- Search users by name or mobile
- Two contact caches for the app's own data: `/api/contact-cache` (string, 15 MB) and `/api/users/cached-contacts` (object)

Privacy & safety

- Privacy settings for last seen, profile photo, about and status (everyone / contacts / nobody), enforced server-side when `PRIVACY_ENFORCE` is on
- Blocking: a blocked contact's messages are never delivered to you; you cannot message someone you blocked
- Report users, messages, groups, channels and statuses (8 reasons)
- View my reports and their moderation status
- Every chat and message endpoint checks that you belong to the chat

1-to-1 chats

- Open or create a chat with any user
- Chat list: pinned first, unread counts, mute/pin/archive flags, latest visible message, pagination — served from cache
- Unread-only view with totals for badges
- Mute for 8 h / 1 day / 1 week / forever — silences pushes
- Pin, archive

- Clear chat for me
- Delete chat for me; reappears on a new message; removed permanently when everyone deleted it
- Chat info with shared media grouped by type

Groups

- Create with icon, description and members; system message announces it
- Members edit name, description, icon and add members
- Admins remove members; the owner cannot be removed
- Leave; admin role passes on automatically; last member deletes the group
- Owner (`groupAdmin`) plus promoted admins; promote and demote
- Lock so only admins can post; unlock
- Delete group with all messages and files (owner)
- Group info: members, admins, shared media

Messaging

- Text, emoji, image, video, audio, document
- Attachments via upload-then-send
- Replies (quoted message returned populated)
- Edit your own messages (saved; marked `isEdited`)
- Forward many messages to many chats
- Delete for everyone (own messages), delete for me (one or many)
- Star / unstar, starred list
- Reactions: one per user, add / change / remove
- Tags on messages
- Delivered and read receipts (✓ / ✓✓ / blue ✓✓)
- Opening a chat marks all its messages read
- Search inside a chat with surrounding context
- Paginated history
- Real-time delivery plus push notifications when offline

Status (stories)

- Text status with background colour and font
- Image and video status
- Automatic deletion after 24 hours
- View tracking (`viewedBy`)
- My statuses with viewers
- Delete own status
- Visibility follows status privacy when `PRIVACY_ENFORCE` is on

Channels

- Create with icon, category, private flag
- Browse and filter by category (private channels hidden)
- Join private channels by invite link
- Owned and subscribed lists
- Subscribe / unsubscribe with live subscriber count
- Subscriber list (admins)
- Admin-only posting with up to 5 attachments

- View counting and unseen-post count
- Delete posts; update or delete channel
- Real-time posts in the channel room; push to subscribers

Broadcast lists

- Create a named recipient list with icon and description
- Edit, add and remove recipients
- Send one message to all recipients as individual chats
- Live delivered / read stats per recipient
- Broadcast history
- Delete broadcast messages and lists

Real time (Socket.IO)

- Online / offline presence across multiple devices, last seen
- Typing indicators
- Delivered and read receipts pushed to the sender
- Live edits, deletions and reactions
- Live group membership and info changes
- Live status posts
- Live channel posts and deletions
- Voice / video call signalling (WebRTC offer, answer, ICE, end, reject)

Push notifications

- Firebase Cloud Messaging on Android, iOS and web
- Sent only when the recipient has no live socket
- Respects mute and notification settings; silent when sound is off
- Group-aware titles ("Group • Sender") and media previews ("📷 Image")
- Dead tokens removed automatically

Media & platform

- Uploads to S3; files served through short-lived signed URLs
- Built-in stickers
- Redis cache for chat list, users and unread counts
- Per-IP rate limiting
- Health endpoint

4. Common flows

Login

```
1. POST /api/auth/send-otp        { mobile }                          → { userId, isNewUser }
2. POST /api/auth/verify-otp      { userId, otp, fcmToken, deviceType, deviceId } → { token, isNewUser, user }
3. if isNewUser:
   POST /api/auth/complete-profile { name, city }                    (with token)
4. connect Socket.IO with { auth: { token } }                         (section 7)
5. GET  /api/chats                                                    → chat list
```

App start with a saved token

```
GET /api/auth/me   → 200: continue (check user.isProfileComplete)
                   → 401: token expired — go to login
```

Open a chat and send a message

```
POST /api/chats              { userId }            → chat   (1-to-1; returns the existing chat if any)
socket.emit("join-chat", chatId)
GET  /api/messages/:chatId                          → messages (newest first) + marks them read
POST /api/messages           { chatId, content }   → message (show single tick)
socket: "message-delivered-update" / "message-read-update" → update ticks
socket.emit("leave-chat", chatId) when the screen closes
```

Send an image

```
POST /api/users/media        multipart "media"                     → data { url, filename, fileType, fileSize }
POST /api/messages           { chatId, messageType: "image", content: "", attachments: [data] }
display: <BASE>/media/<data.url>
```

Create a group

```
POST /api/chats/group  FormData: name, description, users = JSON.stringify([id1, id2]), groupIcon (file, optional)
   (or JSON: { name, description, users: [id1, id2] })
→ members receive socket "message-received" with the "created group" system message
```

5. Endpoint index

#	METHOD	PATH	ACCESS	PURPOSE
1	POST	`/api/auth/send-otp`	Public	Send OTP
2	POST	`/api/auth/verify-otp`	Public	Verify OTP and log in
3	POST	`/api/auth/resend-otp`	Public	Resend OTP
4	POST	`/api/auth/complete-profile`	Token	Complete profile (new users)
5	GET	`/api/auth/me`	Token	Current user
6	POST	`/api/auth/logout`	Token	Log out this device
7	GET	`/api/users/search`	Token + complete profile	Search users
8	POST	`/api/users/sync-contacts`	Token + complete profile	Find which phone contacts use the app
9	DELETE	`/api/users/delete-account`	Token + complete profile	Delete my account
10	GET	`/api/users/contacts`	Token + complete profile	List in-app contacts
11	POST	`/api/users/contacts`	Token + complete profile	Add in-app contact
12	DELETE	`/api/users/contacts/:userId`	Token + complete profile	Remove contact
13	PUT	`/api/users/contacts/:userId`	Token + complete profile	Rename contact
14	GET	`/api/users/invite-code`	Token + complete profile	Get my invite code
15	GET	`/api/users/invite/:code`	Token	Look up an invite code
16	PUT	`/api/users/profile`	Token	Update profile
17	GET	`/api/users/privacy`	Token + complete profile	Get privacy settings
18	PUT	`/api/users/privacy`	Token + complete profile	Update privacy settings
19	POST	`/api/users/media`	Token	Upload a file (for messages)
20	GET	`/api/users/notifications`	Token + complete profile	Get notification settings
21	PUT	`/api/users/notifications`	Token + complete profile	Update notification settings
22	GET	`/api/users/stickers`	Public	List stickers
23	GET	`/api/users/blocked`	Token + complete profile	List blocked users
24	PUT	`/api/users/block/:userId`	Token + complete profile	Block user
25	PUT	`/api/users/unblock/:userId`	Token + complete profile	Unblock user
26	POST	`/api/users/report/:userId`	Token + complete profile	Report user (simple)
27	GET	`/api/users/cached-contacts`	Token + complete profile	Get cached contacts (object form)
28	PUT	`/api/users/cached-contacts`	Token + complete profile	Save cached contacts (object form)
29	GET	`/api/users/:userId`	Token + complete profile	Get a user's profile
30	POST	`/api/chats`	Token + complete profile	Open or create a 1-to-1 chat
31	GET	`/api/chats`	Token + complete profile	Chat list
32	GET	`/api/chats/groups`	Token + complete profile	Group list
33	GET	`/api/chats/unread`	Token + complete profile	Chats with unread messages
34	POST	`/api/chats/group`	Token + complete profile	Create group
35	PUT	`/api/chats/group/update`	Token + complete profile	Update group
36	PUT	`/api/chats/group/rename`	Token + complete profile	Rename group
37	PUT	`/api/chats/group/add`	Token + complete profile	Add members
38	PUT	`/api/chats/group/remove`	Token + complete profile	Remove member
39	PUT	`/api/chats/group/leave`	Token + complete profile	Leave group

#	METHOD	PATH	ACCESS	PURPOSE
40	PUT	`/api/chats/group/add-admin`	Token + complete profile	Make admin
41	PUT	`/api/chats/group/remove-admin`	Token + complete profile	Remove admin
42	PUT	`/api/chats/group/lock`	Token + complete profile	Lock group
43	PUT	`/api/chats/group/unlock`	Token + complete profile	Unlock group
44	DELETE	`/api/chats/group/:chatId`	Token + complete profile	Delete group
45	GET	`/api/chats/group/:chatId/info`	Token + complete profile	Group info
46	PUT	`/api/chats/:chatId/mute`	Token + complete profile	Mute chat
47	PUT	`/api/chats/:chatId/unmute`	Token + complete profile	Unmute chat
48	PUT	`/api/chats/:chatId/pin`	Token + complete profile	Pin chat
49	PUT	`/api/chats/:chatId/unpin`	Token + complete profile	Unpin chat
50	PUT	`/api/chats/:chatId/archive`	Token + complete profile	Archive chat
51	PUT	`/api/chats/:chatId/unarchive`	Token + complete profile	Unarchive chat
52	GET	`/api/chats/:chatId/info`	Token + complete profile	Chat info (1-to-1)
53	DELETE	`/api/chats/:chatId`	Token + complete profile	Delete chat (for me)
54	DELETE	`/api/chats/:chatId/clear`	Token + complete profile	Clear chat
55	POST	`/api/messages`	Token	Send message
56	POST	`/api/messages/forward`	Token + complete profile	Forward messages
57	GET	`/api/messages/starred`	Token + complete profile	Starred messages
58	DELETE	`/api/messages/delete-for-me`	Token	Delete for me (many)
59	DELETE	`/api/messages`	Token + complete profile	Delete for everyone
60	GET	`/api/messages/:chatId`	Token + complete profile	Get messages (and mark as read)
61	DELETE	`/api/messages/:messageId`	Token	Delete for everyone (path form)
62	PUT	`/api/messages/:messageId`	Token	Edit my message
63	DELETE	`/api/messages/:messageId/delete-for-me`	Token	Delete for me (one)
64	PUT	`/api/messages/:messageId/read`	Token	Mark one message read
65	PUT	`/api/messages/:messageId/star`	Token	Star / unstar
66	PUT	`/api/messages/:messageId/react`	Token	Add or change reaction
67	DELETE	`/api/messages/:messageId/react`	Token	Remove my reaction
68	PUT	`/api/messages/:messageId/tag`	Token	Add tag
69	POST	`/api/status`	Token	Post a status
70	GET	`/api/status`	Token + complete profile	All statuses
71	GET	`/api/status/my`	Token + complete profile	My statuses
72	PUT	`/api/status/:statusId/view`	Token	Mark status viewed
73	DELETE	`/api/status/:statusId`	Token	Delete my status
74	POST	`/api/channels`	Token + complete profile	Create channel
75	GET	`/api/channels`	Token + complete profile	Browse channels
76	GET	`/api/channels/my/owned`	Token + complete profile	Channels I own
77	GET	`/api/channels/my/subscribed`	Token + complete profile	Channels I follow
78	PUT	`/api/channels/join/:inviteLink`	Token + complete profile	Join by invite link
79	GET	`/api/channels/:channelId`	Token + complete profile	Channel details
80	PUT	`/api/channels/:channelId/update`	Token + complete profile	Update channel

#	METHOD	PATH	ACCESS	PURPOSE
81	DELETE	`/api/channels/:channelId`	Token + complete profile	Delete channel
82	PUT	`/api/channels/:channelId/subscribe`	Token + complete profile	Subscribe
83	PUT	`/api/channels/:channelId/unsubscribe`	Token + complete profile	Unsubscribe
84	GET	`/api/channels/:channelId/subscribers`	Token + complete profile	Subscribers
85	POST	`/api/channels/:channelId/messages`	Token + complete profile	Post to channel
86	GET	`/api/channels/:channelId/messages`	Token + complete profile	Channel posts
87	GET	`/api/channels/:channelId/unseen-count`	Token + complete profile	Unseen posts
88	DELETE	`/api/channels/:channelId/messages/:messageId`	Token + complete profile	Delete post
89	POST	`/api/broadcasts`	Token + complete profile	Create broadcast list
90	GET	`/api/broadcasts`	Token + complete profile	My broadcast lists
91	GET	`/api/broadcasts/:broadcastId`	Token + complete profile	Broadcast list details
92	PUT	`/api/broadcasts/:broadcastId`	Token + complete profile	Update broadcast list
93	DELETE	`/api/broadcasts/:broadcastId`	Token + complete profile	Delete broadcast list
94	PUT	`/api/broadcasts/:broadcastId/add-recipients`	Token + complete profile	Add recipients
95	PUT	`/api/broadcasts/:broadcastId/remove-recipients`	Token + complete profile	Remove recipients
96	POST	`/api/broadcasts/:broadcastId/messages`	Token + complete profile	Send broadcast
97	GET	`/api/broadcasts/:broadcastId/messages`	Token + complete profile	Sent broadcast messages
98	GET	`/api/broadcasts/messages/:messageId/stats`	Token + complete profile	Broadcast delivery stats
99	DELETE	`/api/broadcasts/messages/:messageId`	Token	Delete broadcast message
100	POST	`/api/reports/user`	Token	Report user
101	POST	`/api/reports/message`	Token	Report message
102	POST	`/api/reports/group`	Token	Report group
103	POST	`/api/reports/channel`	Token	Report channel
104	POST	`/api/reports/status`	Token	Report status
105	GET	`/api/reports/my`	Token	My reports
106	GET	`/api/contact-cache`	Token + complete profile	Get contact cache
107	PUT	`/api/contact-cache`	Token + complete profile	Save contact cache
108	DELETE	`/api/contact-cache`	Token + complete profile	Delete contact cache
109	POST	`/api/notifications/save-token`	Token	Register / refresh FCM token
110	POST	`/api/test-notification/send-test-notification`	Token	Send a test push
111	GET	`/media/*`	Public	Open an uploaded file
112	GET	`/health`	Public	Health check

6. Endpoints

6.1 Auth

Phone-number + OTP login. There is no password. A JWT (valid 30 days) is returned by verify-otp and sent as `Authorization: Bearer <token>` on every private call.

POST /api/auth/send-otp — Send OTP

Access: Public · **Body:** `application/json`

Starts login. Creates the user if the number is new. Sends an OTP by SMS (valid 10 minutes). Use `isNewUser` to decide the next screen after verification.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`mobile`	string	yes	Phone number. Non-digits are stripped and the last 10 digits are used, so `+91 98765-43210` works.
`isdCode`	string	no	Country code, default `"91"`.

Example — New user

```
POST /api/auth/send-otp
Content-Type: application/json

{
  "mobile": "9000000001"
}
```

Response `200`

```
{
  "success": true,
  "message": "OTP sent to 9000000001",
  "userId": "6aaceedb5a7dcc4c6e188876",
  "isNewUser": true
}
```

Example — Returning user

```
POST /api/auth/send-otp
Content-Type: application/json

{
  "mobile": "9000000001"
}
```

Response `200`

```
{
  "success": true,
  "message": "OTP sent to 9000000001",
  "userId": "6aaceedb5a7dcc4c6e188876",
  "isNewUser": false
}
```

Error responses

STATUS	MESSAGE	WHEN
400	Mobile number is required	
400	Please provide a valid 10-digit mobile number	
500	Failed to send OTP. Please try again.	
500	Server error	

Notes

- When the server runs with SMS disabled, the OTP is always `1234` (development only).

`POST /api/auth/verify-otp` — Verify OTP and log in

Access: Public · **Body:** `application/json`

Verifies the OTP and returns the JWT. Marks the user verified. Send this device's FCM token here so push notifications work from the first login. The token is removed from any other account it was attached to.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`userId`	ObjectId	yes	From send-otp.
`otp`	string	yes	4-digit code.
`fcmToken`	string	no	`messaging().getToken()` of this device.
`deviceType`	string	with fcmToken	`android` | `ios` | `web`.
`deviceId`	string	no	Stable id of this install. Replaces this device's previous token instead of adding a duplicate.

Example — With FCM token (recommended)

```
POST /api/auth/verify-otp
Content-Type: application/json

{
  "userId": "6aaceee45a7dcc4c6e188899",
  "otp": "1234",
  "fcmToken": "fcm-device-token-from-firebase",
  "deviceType": "android",
  "deviceId": "a1b2c3d4-install-id"
}
```

Response `200`

```
{
  "success": true,
  "message": "OTP verified successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5…(JWT)",
  "isNewUser": true,
  "user": {
    "id": "6aaceee45a7dcc4c6e188899",
    "mobile": "9000000007",
    "name": null
  }
}
```

Example

```
POST /api/auth/verify-otp
Content-Type: application/json

{
  "userId": "6aaceedb5a7dcc4c6e188876",
  "otp": "1234"
}
```

Response `200`

```
{
  "success": true,
  "message": "OTP verified successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5…(JWT)",
  "isNewUser": true,
  "user": {
    "id": "6aaceedb5a7dcc4c6e188876",
    "mobile": "9000000001",
    "name": null
  }
}
```

Error responses

STATUS	MESSAGE	WHEN
400	Invalid OTP	Wrong OTP
400	userId and otp are required	
404	User not found	
400	OTP expired	
500	Server error	

Notes

- `isNewUser: true` means the profile is empty — go to the profile-completion screen and call `POST /api/auth/complete-profile`. Every endpoint with access "Token + complete profile" returns 403 until then.
- Also send the token on every later app start with `POST /api/notifications/save-token` — see section 8.

`POST /api/auth/resend-otp` — Resend OTP

Access: Public · **Body:** `application/json`

Generates and sends a new OTP for an existing user id.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`userId`	ObjectId	yes	From send-otp.

Example

```
POST /api/auth/resend-otp
Content-Type: application/json

{
  "userId": "6aaceedb5a7dcc4c6e188876"
}
```

Response `200`

```
{
  "success": true,
  "message": "OTP resent successfully"
}
```

Error responses

STATUS	MESSAGE	WHEN
400	userId is required	
404	User not found	
500	Failed to send OTP.	
500	Server error	

`POST /api/auth/complete-profile` — Complete profile (new users)

Access: Token · **Body:** `application/json`

One-time step after the first login. Sets name and city. Fails if the profile is already complete — use `PUT /api/users/profile` for later edits.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`name`	string	yes	Max 50 characters.
`city`	string	yes	

Example

```
POST /api/auth/complete-profile
Authorization: Bearer <token>
Content-Type: application/json

{
  "name": "Rahul Sharma",
  "city": "Indore"
}
```

Response `200`

```
{
  "success": true,
  "message": "Profile completed! Welcome to the app 🎉",
  "user": {
    "id": "6aaceedb5a7dcc4c6e188876",
    "name": "Rahul Sharma",
    "mobile": "9000000001",
    "city": "Indore",
    "profilePicture": "",
    "about": "Hey there! I am using TeleBook",
    "isProfileComplete": true
  }
}
```

Error responses

STATUS	MESSAGE	WHEN
400	Profile already completed. Use update-profile to change details.	Already completed
400	Missing required fields: city	Missing field
400	Missing required fields: ${missing.join(', ')}	
500	Server error	

`GET /api/auth/me` — Current user

Access: Token · **Body:** —

Returns the logged-in user's full profile plus `isProfileComplete`. Use on app start to restore the session.

Example

```
GET /api/auth/me
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "user": {
    "privacySettings": {
      "lastSeen": "everyone",
      "profilePhoto": "everyone",
      "about": "everyone",
      "status": "contacts"
    },
    "notificationSettings": {
      "messageNotifications": true,
      "groupNotifications": true,
      "statusNotifications": true,
      "channelNotifications": true,
      "sound": true,
      "vibration": true
    },
    "_id": "6aaceedb5a7dcc4c6e188876",
    "name": "Rahul Sharma",
    "mobile": "9000000001",
    "district": null,
    "city": "Indore",
    "state": null,
    "profilePicture": "",
    "about": "Hey there! I am using TeleBook",
    "isOnline": false,
    "socketId": "",
    "isVerified": true,
    "isDeleted": false,
    "blockedUsers": [],
    "isdCode": "91",
    "lastSeen": "2026-09-18T07:57:15.923Z",
    "savedContacts": [],
    "reportedUsers": [],
    "fcmTokens": [],
    "createdAt": "2026-09-18T07:57:15.940Z",
    "updatedAt": "2026-09-18T07:57:27.050Z",
    "__v": 0,
    "isProfileComplete": true
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
401	Not authorized – no token provided	No token

`POST /api/auth/logout` — Log out this device

Access: Token · **Body:** `application/json`

Removes this device's FCM token from the account so it stops receiving push notifications. The JWT cannot be revoked by the server — delete it in the app.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`deviceId`	string	one of the two	The same `deviceId` sent at login.
`fcmToken`	string	one of the two	The device's current token.

Example

```
POST /api/auth/logout
Authorization: Bearer <token>
Content-Type: application/json

{
  "deviceId": "a1b2c3d4-install-id"
}
```

Response `200`

```
{
  "success": true,
  "message": "Logged out",
  "removedDevices": 1
}
```

Error responses

STATUS	MESSAGE	WHEN
400	deviceId or fcmToken is required	Missing deviceId and fcmToken
500	Server error	

Notes

- Call before `messaging().deleteToken()` and before clearing the saved JWT.

6.2 Users & Profile

Profile, in-app contact book, phone-contact sync, privacy, notification settings, block list, invite codes, media upload and account deletion.

`GET /api/users/search` — Search users

Access: Token + complete profile · **Body:** —

A 6–10 digit query is matched exactly against mobile numbers; anything else is a case-insensitive name search. Returns at most 20 verified users, never yourself.

Query parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`query`	string	yes	Name or mobile.

Example — By name

```
GET /api/users/search?query=Priya
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "users": [
    {
      "_id": "6aaceedd5a7dcc4c6e18887d",
      "name": "Priya Verma",
      "mobile": "9000000002",
      "district": null,
      "city": "Bhopal",
      "state": null,
      "profilePicture": "",
      "about": "Hey there! I am using TeleBook",
      "isOnline": false,
      "lastSeen": "2026-09-18T07:57:17.878Z"
    }
  ]
}
```


Example — By mobile

```
GET /api/users/search?query=9000000003
Authorization: Bearer <token>
```

Response `200`


```
{
  "success": true,
  "users": [
    {
      "_id": "6aaceedf5a7dcc4c6e188884",
      "name": "Amit Kumar",
      "mobile": "9000000003",
      "district": null,
      "city": "Indore",
      "state": null,
      "profilePicture": "",
      "about": "Hey there! I am using TeleBook",
      "isOnline": false,
      "lastSeen": "2026-09-18T07:57:19.708Z"
    }
  ]
}
```


Error responses

STATUS	MESSAGE	WHEN
400	Query is required	

POST /api/users/sync-contacts — Find which phone contacts use the app

Access: Token + complete profile · **Body:** `application/json`

Send the phone book's numbers; returns the ones registered and verified on the app. Nothing is stored.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`contacts`	array	yes	Phone numbers in any format; last 10 digits are used.

Example

```
POST /api/users/sync-contacts
Authorization: Bearer <token>
Content-Type: application/json

{
  "contacts": [
    "+91 90000 00002",
    "9000000003",
    "…1 more"
  ]
}
```

Response `200`


```
{
  "success": true,
  "total": 3,
  "found": 2,
  "users": [
    {
      "_id": "6aaceedd5a7dcc4c6e18887d",
      "name": "Priya Verma",
      "mobile": "9000000002",
      "profilePicture": "",
      "about": "Hey there! I am using TeleBook",
      "isOnline": false,
      "lastSeen": "2026-09-18T07:57:17.878Z"
    },
    {
      "_id": "6aaceedf5a7dcc4c6e188884",
      "name": "Amit Kumar",
      "mobile": "9000000003",
      "profilePicture": "",
      "about": "Hey there! I am using TeleBook",
      "isOnline": false,
      "lastSeen": "2026-09-18T07:57:19.708Z"
    }
  ]
}
```


Error responses

STATUS	MESSAGE	WHEN
400	contacts array is required	

Notes

- `total` = valid numbers sent, `found` = registered users returned.

`DELETE /api/users/delete-account` — Delete my account

Access: Token + complete profile · **Body:** —

Removes you from every group (passing ownership on if you were the only owner) and channel, deletes your statuses, contact cache and profile picture, removes you from other users' contacts and block lists, and anonymises the account: it stays in existing 1-to-1 chats as **"Deleted account"** so other users' chats keep working. The phone number is freed and can register again. Your token stops working immediately. Cannot be undone.

Example

```
DELETE /api/users/delete-account
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Account deleted successfully"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	User not found	
500	Server error	

`GET /api/users/contacts` — List in-app contacts

Access: Token + complete profile · **Body:** —

Contacts saved inside the app (not the phone book), with presence.

Query parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`page`	number	no	Default 1.
`limit`	number	no	Default 50.

Example

```
GET /api/users/contacts?page=1&limit=20
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "contacts": [
    {
      "_id": "6aaceedd5a7dcc4c6e18887d",
      "displayName": "Priya Office",
      "mobile": "9000000002",
      "profilePicture": "",
      "about": "Hey there! I am using TeleBook",
      "isOnline": false,
      "lastSeen": "2026-09-18T07:57:17.878Z",
      "addedAt": "2026-09-18T07:57:30.476Z"
    },
    {
      "_id": "6aaceedf5a7dcc4c6e188884",
      "displayName": "Amit Kumar",
      "mobile": "9000000003",
      "profilePicture": "",
      "about": "Hey there! I am using TeleBook",
      "isOnline": false,
      "lastSeen": "2026-09-18T07:57:19.708Z",
      "addedAt": "2026-09-18T07:57:32.044Z"
    }
  ],
  "pagination": {
    "currentPage": 1,
    "totalPages": 1,
    "totalContacts": 2,
    "hasMore": false,
    "limit": 20
  }
}
```

`POST /api/users/contacts` — Add in-app contact

Access: Token + complete profile · **Body:** `application/json`

Adds a registered user to your in-app contacts by mobile number.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`mobile`	string	yes	10-digit number of a registered user.
`displayName`	string	no	Your own name for them.

Example

```
POST /api/users/contacts
Authorization: Bearer <token>
Content-Type: application/json

{
  "mobile": "9000000002",
  "displayName": "Priya Office"
}
```

Response `201`

```
{
  "success": true,
  "message": "Contact added successfully",
  "contact": {
    "_id": "6aaceedd5a7dcc4c6e18887d",
    "displayName": "Priya Office",
    "mobile": "9000000002",
    "profilePicture": "",
    "isOnline": false
  }
}
```

Error responses

STATUS	MESSAGE	WHEN
400	mobile is required	
404	No registered user found with this mobile number	
400	You cannot add yourself	
400	Contact already saved	

`DELETE /api/users/contacts/:userId` — Remove contact

Access: Token + complete profile · **Body:** —

Removes a user from your in-app contacts.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`userId`	ObjectId	yes	

Example

```
DELETE /api/users/contacts/6aaceedf5a7dcc4c6e188884
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Contact removed"
}
```

`PUT /api/users/contacts/:userId` — Rename contact

Access: Token + complete profile · **Body:** `application/json`

Sets your display name for a saved contact.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`userId`	ObjectId	yes	The contact.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`displayName`	string	yes	

Example

```
PUT /api/users/contacts/6aaceedd5a7dcc4c6e18887d
Authorization: Bearer <token>
Content-Type: application/json

{
  "displayName": "Priya (Work)"
}
```

Response `200`

```
{
  "success": true,
  "message": "Contact renamed",
  "displayName": "Priya (Work)"
}
```

Error responses

STATUS	MESSAGE	WHEN
400	displayName required	
404	Contact not found	

`GET /api/users/invite-code` — Get my invite code

Access: Token + complete profile · **Body:** —

Returns your personal invite code, creating it on first call.

Example

```
GET /api/users/invite-code
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "inviteCode": "219dab9bd3e0",
  "inviteLink": "telebookservices.com/join/219dab9bd3e0"
}
```

Notes

- `inviteLink` is built from the server's `APP_BASE_URL` setting. Until it is set, the link uses the placeholder `https://yourapp.com`; build the link in the app from `inviteCode` if needed.

`GET /api/users/invite/:code` — Look up an invite code

Access: Token · **Body:** —

Returns the user who owns an invite code. Works before the profile is complete.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`code`	string	yes	

Example

```
GET /api/users/invite/219dab9bd3e0
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "user": {
    "_id": "6aaceedb5a7dcc4c6e188876",
    "name": "Rahul Sharma",
    "mobile": "9000000001",
    "profilePicture": "",
    "about": "Busy right now",
    "isOnline": false
  }
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Invalid invite code	

`PUT /api/users/profile` — Update profile

Access: Token · **Body:** `multipart/form-data` (JSON also works when no file is sent)

Updates any of the given fields. Only non-empty values are applied. Replacing the picture deletes the old file from storage.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`name`	string	no	
`about`	string	no	Max 139 characters.
`district`	string	no	
`city`	string	no	

FIELD	TYPE	REQUIRED	DESCRIPTION
`state`	string	no	

Files

FIELD	TYPE	REQUIRED	DESCRIPTION
`profilePicture`	image	no	Multipart field name.

Example — Text fields (JSON)

```
PUT /api/users/profile
Authorization: Bearer <token>
Content-Type: application/json

{
  "about": "Busy right now",
  "district": "Indore",
  "state": "Madhya Pradesh"
}
```

Response `200`


```
{
  "success": true,
  "user": {
    "privacySettings": {
      "lastSeen": "everyone",
      "profilePhoto": "everyone",
      "about": "everyone",
      "status": "contacts"
    },
    "notificationSettings": {
      "messageNotifications": true,
      "groupNotifications": true,
      "statusNotifications": true,
      "channelNotifications": true,
      "sound": true,
      "vibration": true
    },
    "_id": "6aaceedb5a7dcc4c6e188876",
    "name": "Rahul Sharma",
    "mobile": "9000000001",
    "district": "Indore",
    "city": "Indore",
    "state": "Madhya Pradesh",
    "profilePicture": "",
    "about": "Busy right now",
    "isOnline": false,
    "socketId": "",
    "isVerified": true,
    "isDeleted": false,
    "blockedUsers": [],
    "isdCode": "91",
    "lastSeen": "2026-09-18T07:57:15.923Z",
    "savedContacts": [],
    "reportedUsers": [],
    "fcmTokens": [],
    "createdAt": "2026-09-18T07:57:15.940Z",
    "updatedAt": "2026-09-18T07:57:28.358Z",
    "__v": 0
  }
}
```


Notes

- Without a picture, plain JSON works.
- `profilePicture` in the response is a storage key, not a URL — display it through `GET /media/<key>`.

`GET /api/users/privacy` — Get privacy settings

Access: Token + complete profile · **Body:** —

Your visibility settings.

Example

```
GET /api/users/privacy
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "privacySettings": {
    "lastSeen": "everyone",
    "profilePhoto": "everyone",
    "about": "everyone",
    "status": "contacts"
  }
}
```

`PUT /api/users/privacy` — Update privacy settings

Access: Token + complete profile · **Body:** `application/json`

Each field accepts `everyone`, `contacts` or `nobody`. Unknown values are ignored. **Contacts** means:
people the user saved in the app plus everyone they have a 1-to-1 chat with.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`lastSeen`	string	no	
`profilePhoto`	string	no	
`about`	string	no	
`status`	string	no	

Example

```
PUT /api/users/privacy
Authorization: Bearer <token>
Content-Type: application/json

{
  "lastSeen": "contacts",
  "profilePhoto": "everyone"
}
```

Response `200`

```
{
  "success": true,
  "privacySettings": {
    "lastSeen": "contacts",
    "profilePhoto": "everyone",
    "about": "everyone",
    "status": "contacts"
  }
}
```

Notes

- Enforced only when the server runs with `PRIVACY_ENFORCE=true` (see Server configuration). When enforced, a hidden `lastSeen` arrives as `null` with `isOnline: false`, and a hidden `profilePicture` or `about` arrives as `""` — the field is always present.

`POST /api/users/media` — Upload a file (for messages)

Access: Token · **Body:** `multipart/form-data`

Uploads one file to storage and returns its metadata. Put the returned object into `attachments` of `POST /api/messages`.

Files

FIELD	TYPE	REQUIRED	DESCRIPTION
`media`	any file	yes	Multipart field name. Max 100 MB.

Example

```
POST /api/users/media
Authorization: Bearer <token>
Content-Type: multipart/form-data

media: <file: photo.png, image/png>
```

Response `200`

```
{
  "success": true,
  "message": "File uploaded successfully",
  "data": {
    "url": "uploads/messages/images/media-1789718248836-13397368.png",
    "filename": "photo.png",
    "fileType": "image/png",
    "fileSize": 70
  }
}
```

Notes

- Two-step send: upload here first, then send the message with `attachments: [data]`.

`GET /api/users/notifications` — Get notification settings

Access: Token + complete profile · **Body:** —

Your notification preferences.

Example

```
GET /api/users/notifications
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "notificationSettings": {
    "messageNotifications": true,
    "groupNotifications": true,
    "statusNotifications": true,
    "channelNotifications": true,
    "sound": true,
    "vibration": true
  }
}
```

`PUT /api/users/notifications` — Update notification settings

Access: Token + complete profile · **Body:** `application/json`

Send only the keys to change. Applied to push notifications: `messageNotifications` (1-to-1), `groupNotifications` (groups), `channelNotifications` (channel posts); `sound: false` sends silent pushes.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageNotifications`	boolean	no	
`groupNotifications`	boolean	no	
`statusNotifications`	boolean	no	Stored; statuses do not push.
`channelNotifications`	boolean	no	
`sound`	boolean	no	
`vibration`	boolean	no	Stored; controlled by the device notification channel.

Example

```
PUT /api/users/notifications
Authorization: Bearer <token>
Content-Type: application/json

{
  "groupNotifications": false,
  "vibration": false
}
```

Response `200`

```
{
  "success": true,
  "notificationSettings": {
    "messageNotifications": true,
    "groupNotifications": false,
    "statusNotifications": true,
    "channelNotifications": true,
    "sound": true,
    "vibration": false
  }
}
```

GET /api/users/stickers — List stickers

Access: Public · **Body:** —

Built-in stickers available to send. Public — no token needed.

Example

```
GET /api/users/stickers
```

Response `200`

```
{
  "success": true,
  "message": "Stickers fetched successfully",
  "data": [
    {
      "url": "uploads/stickers/cancle.png",
      "filename": "cancle.png",
      "fileType": "png",
      "fileSize": 42114
    },
    {
      "url": "uploads/stickers/delete.png",
      "filename": "delete.png",
      "fileType": "png",
      "fileSize": 61683
    },
    "…3 more"
  ]
}
```

Notes

- `url` is a server path — load it as `<BASE_URL>/<url>` (served statically from `/uploads`).

GET /api/users/blocked — List blocked users

Access: Token + complete profile · **Body:** —

Users you have blocked.

Example

```
GET /api/users/blocked
Authorization: Bearer <token>
```

Response `200`


```
{
  "success": true,
  "blockedUsers": [
    {
      "_id": "6aaceee15a7dcc4c6e18888b",
      "name": "Sneha Patel",
      "mobile": "9000000004",
      "profilePicture": ""
    }
  ]
}
```

PUT /api/users/block/:userId — Block user

Access: Token + complete profile · **Body:** `application/json`

Adds the user to your block list.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`userId`	ObjectId	yes	

Example

```
PUT /api/users/block/6aaceee15a7dcc4c6e18888b
Authorization: Bearer <token>
```

Response `200`


```
{
  "success": true,
  "message": "User blocked"
}
```

PUT /api/users/unblock/:userId — Unblock user

Access: Token + complete profile · **Body:** `application/json`

Removes the user from your block list.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`userId`	ObjectId	yes	

Example

```
PUT /api/users/unblock/6aaceee15a7dcc4c6e18888b
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "User unblocked"
}
```

`POST /api/users/report/:userId` — Report user (simple)

Access: Token + complete profile · **Body:** `application/json`

Records a report on your own user record. For the moderation queue use `POST /api/reports/user` instead.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`userId`	ObjectId	yes	

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`reason`	string	no	Free text.

Example

```
POST /api/users/report/6aaceee15a7dcc4c6e18888b
Authorization: Bearer <token>
Content-Type: application/json

{
  "reason": "spam"
}
```

Response `200`

```
{
  "success": true,
  "message": "User reported successfully"
}
```

Error responses

STATUS	MESSAGE	WHEN
400	Reason is required	
400	Already reported	

`GET /api/users/cached-contacts` — Get cached contacts (object form)

Access: Token + complete profile · **Body:** —

Returns the object stored by the PUT below. See also `/api/contact-cache`, which is a separate store.

Example

```
GET /api/users/cached-contacts
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "data": {
    "lastSync": "2026-09-18T10:00:00Z",
    "count": 2
  }
}
```

Error responses

STATUS	MESSAGE	WHEN
404	User not found	
500	Failed to retrieve cached contacts data	

`PUT /api/users/cached-contacts` — Save cached contacts (object form)

Access: Token + complete profile · **Body:** `application/json`

Stores any JSON **object** on your user record and returns it.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`contactsData`	object	yes	Must be a JSON object, not an array or string.

Example

```
PUT /api/users/cached-contacts
Authorization: Bearer <token>
Content-Type: application/json

{
  "contactsData": {
    "lastSync": "2026-09-18T10:00:00Z",
    "count": 2
  }
}
```

Response `200`

```
{
  "success": true,
  "message": "Cached contacts data saved successfully",
  "data": {
    "lastSync": "2026-09-18T10:00:00Z",
    "count": 2
  }
}
```

Error responses

STATUS	MESSAGE	WHEN
400	contactsData is required	
400	contactsData must be a valid JSON object	
500	Failed to save cached contacts data	

Notes

- Two separate contact stores exist: this one (object, on the user record, no size check) and `/api/contact-cache` (string, own collection, size-checked). They do not share data — pick one and use it for both reading and writing.

`GET /api/users/:userId` — Get a user's profile

Access: Token + complete profile · **Body:** —

Public profile of another user: `_id`, `name`, `mobile`, `profilePicture`, `about`, `isOnline`, `lastSeen`, `district`, `city`, `state`. A deleted account returns `name: "Deleted account"` with empty fields.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`userId`	ObjectId	yes	

Example

```
GET /api/users/6aaceedd5a7dcc4c6e18887d
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "user": {
    "_id": "6aaceedd5a7dcc4c6e18887d",
    "name": "Priya Verma",
    "mobile": "9000000002",
    "profilePicture": "",
    "about": "Hey there! I am using TeleBook",
    "isOnline": false,
    "lastSeen": "2026-09-18T07:57:17.878Z",
    "district": null,
    "city": "Bhopal",
    "state": null
  }
}
```

Error responses

STATUS	MESSAGE	WHEN
404	User not found	

6.3 Chats

One-to-one chats and groups: listing, creation, group administration and per-user chat state (mute, pin, archive, clear, delete).

`POST /api/chats` — Open or create a 1-to-1 chat

Access: Token + complete profile · **Body:** `application/json`

Returns the existing chat with this user (200) or creates one (201). Call this before sending the first message.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`userId`	ObjectId	yes	The other person.

Example — New chat (201)

```
POST /api/chats
Authorization: Bearer <token>
Content-Type: application/json

{
  "userId": "6aaceedd5a7dcc4c6e18887d"
}
```

Response `201`

```
{
  "success": true,
  "chat": {
    "_id": "6aaceef55a7dcc4c6e1888e2",
    "chatName": "sender",
    "isGroupChat": false,
    "users": [
      {
        "privacySettings": {
          "lastSeen": "contacts",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": false,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": false
        },
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "mobile": "9000000001",
        "district": "Indore",
        "city": "Indore",
        "state": "Madhya Pradesh",
        "profilePicture": "",
        "about": "Busy right now",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:15.923Z",
        "createdAt": "2026-09-18T07:57:15.940Z",
        "updatedAt": "2026-09-18T07:57:40.250Z",
        "__v": 4,
        "inviteCode": "219dab9bd3e0"
      },
      {
        "privacySettings": {
          "lastSeen": "everyone",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": true,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": true
        },
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "district": null,
        "city": "Bhopal",
        "state": null,
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:17.878Z",
        "createdAt": "2026-09-18T07:57:17.881Z",
        "updatedAt": "2026-09-18T07:57:19.144Z",
```

```
        "__v": 0
      }
    ],
    "groupAdmin": [],
    "groupIcon": "",
    "groupDescription": "",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:41.363Z",
    "updatedAt": "2026-09-18T07:57:41.363Z",
    "__v": 0
  }
}
```


Example — Existing chat (200)

```
POST /api/chats
Authorization: Bearer <token>
Content-Type: application/json

{
  "userId": "6aaceedd5a7dcc4c6e18887d"
}
```

Response `200`

```
{
  "success": true,
  "chat": {
    "_id": "6aaceef55a7dcc4c6e1888e2",
    "chatName": "sender",
    "isGroupChat": false,
    "users": [
      {
        "privacySettings": {
          "lastSeen": "contacts",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": false,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": false
        },
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "mobile": "9000000001",
        "district": "Indore",
        "city": "Indore",
        "state": "Madhya Pradesh",
        "profilePicture": "",
        "about": "Busy right now",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:15.923Z",
        "createdAt": "2026-09-18T07:57:15.940Z",
        "updatedAt": "2026-09-18T07:57:40.250Z",
        "__v": 4,
        "inviteCode": "219dab9bd3e0"
      },
      {
        "privacySettings": {
          "lastSeen": "everyone",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": true,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": true
        },
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "district": null,
        "city": "Bhopal",
        "state": null,
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:17.878Z",
        "createdAt": "2026-09-18T07:57:17.881Z",
        "updatedAt": "2026-09-18T07:57:19.144Z",
```

```
        "__v": 0
      }
    ],
    "groupAdmin": [],
    "groupIcon": "",
    "groupDescription": "",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:41.363Z",
    "updatedAt": "2026-09-18T07:57:41.363Z",
    "__v": 0
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	UserId is required	

`GET /api/chats` — Chat list

Access: Token + complete profile · **Body:** —

Your chats, pinned first, then by latest activity. Each chat includes `unseenCount`, `isPinned`, `isMuted`, `isArchived`, populated `users`, `groupAdmin` and `latestMessage`. Chats you deleted are hidden until a new message arrives. `latestMessage` is the last message **you** can see — messages you deleted for yourself, cleared, or received from someone you blocked are skipped; the key is absent when none is left.

Query parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`page`	number	no	Default 1.
`limit`	number	no	Default 50.

Example

```
GET /api/chats?page=1&limit=20
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "chats": [
    {
      "_id": "6aaceef55a7dcc4c6e1888e2",
      "chatName": "sender",
      "isGroupChat": false,
      "users": [
        {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": "",
          "mobile": "9000000001",
          "isOnline": false,
          "lastSeen": "2026-09-18T07:57:15.923Z"
        },
        {
          "_id": "6aaceedd5a7dcc4c6e18887d",
          "name": "Priya Verma",
          "profilePicture": "",
          "mobile": "9000000002",
          "isOnline": false,
          "lastSeen": "2026-09-18T07:57:17.878Z"
        }
      ],
      "groupAdmin": [],
      "groupIcon": "",
      "groupDescription": "",
      "deletedBy": [],
      "pinnedBy": [],
      "archivedBy": [],
      "admins": [],
      "isLocked": false,
      "mutedBy": [],
      "createdAt": "2026-09-18T07:57:41.363Z",
      "updatedAt": "2026-09-18T07:58:14.451Z",
      "__v": 0,
      "latestMessage": {
        "_id": "6aacef155a7dcc4c6e18899b",
        "sender": {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        },
        "content": "This message was deleted",
        "chat": "6aaceef55a7dcc4c6e1888e2",
        "messageType": "text",
        "attachments": [],
        "isDeleted": true,
        "deletedFor": [],
        "isEdited": false,
        "isForwarded": false,
        "starredBy": [],
        "tags": [],
        "isBroadcast": false,
        "readBy": [],
        "deliveredTo": [],
        "reactions": [],
        "createdAt": "2026-09-18T07:58:13.553Z",
        "updatedAt": "2026-09-18T07:58:16.300Z",
        "__v": 0
      },
      "isPinned": false,
      "isMuted": false,
      "isArchived": false,
      "unseenCount": 4
    },
    {
      "_id": "6aaceef65a7dcc4c6e1888e8",
      "chatName": "Weekend Trip",
      "isGroupChat": true,
      "users": [
```

```
        {
          "_id": "6aaceedd5a7dcc4c6e18887d",
          "name": "Priya Verma",
          "profilePicture": "",
          "mobile": "9000000002",
          "isOnline": false,
          "lastSeen": "2026-09-18T07:57:17.878Z"
        },
        {
          "_id": "6aaceedf5a7dcc4c6e188884",
          "name": "Amit Kumar",
          "profilePicture": "",
          "mobile": "9000000003",
          "isOnline": false,
          "lastSeen": "2026-09-18T07:57:19.708Z"
        },
        "…1 more"
      ],
      "groupAdmin": [
        {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        }
      ],
      "groupIcon": "",
      "groupDescription": "Plan for Sunday",
      "deletedBy": [],
      "pinnedBy": [],
      "archivedBy": [],
      "admins": [],
      "isLocked": false,
      "mutedBy": [],
      "createdAt": "2026-09-18T07:57:42.731Z",
      "updatedAt": "2026-09-18T07:58:07.711Z",
      "__v": 0,
      "latestMessage": {
        "_id": "6aacef0f5a7dcc4c6e188976",
        "sender": {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        },
        "content": "Hello Rahul",
        "chat": "6aaceef65a7dcc4c6e1888e8",
        "messageType": "text",
        "attachments": [],
        "isDeleted": false,
        "deletedFor": [],
        "isEdited": false,
        "isForwarded": true,
        "forwardedFrom": "6aaceedd5a7dcc4c6e18887d",
        "starredBy": [],
        "tags": [],
        "isBroadcast": false,
        "readBy": [],
        "deliveredTo": [],
        "reactions": [],
        "createdAt": "2026-09-18T07:58:07.050Z",
        "updatedAt": "2026-09-18T07:58:07.050Z",
        "__v": 0
      },
      "isPinned": false,
      "isMuted": false,
      "isArchived": false,
      "unseenCount": 3
    },
    "…1 more"
  ],
  "pagination": {
    "currentPage": 1,
    "totalPages": 1,
```

```
    "totalChats": 3,
    "hasMore": false,
    "limit": 20
  }
}
```

Error responses

STATUS	MESSAGE	WHEN
403	Please complete your profile first (name, district, city, state)	Profile not completed

Notes

- Served from a Redis cache, kept current on every change. Refetching is cheap.
- Archived chats are included — filter on `isArchived` for an archive view.

`GET /api/chats/groups` — Group list

Access: Token + complete profile · **Body:** —

Only group chats you belong to.

Query parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`page`	number	no	
`limit`	number	no	

Example

```
GET /api/chats/groups
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "count": 2,
  "groups": [
    {
      "_id": "6aaceef65a7dcc4c6e1888e8",
      "chatName": "Weekend Trip",
      "isGroupChat": true,
      "users": [
        {
          "_id": "6aaceedd5a7dcc4c6e18887d",
          "name": "Priya Verma",
          "mobile": "9000000002",
          "profilePicture": ""
        },
        {
          "_id": "6aaceedf5a7dcc4c6e188884",
          "name": "Amit Kumar",
          "mobile": "9000000003",
          "profilePicture": ""
        },
        "…1 more"
      ],
      "groupAdmin": [
        {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        }
      ],
      "groupIcon": "",
      "groupDescription": "Plan for Sunday",
      "deletedBy": [],
      "pinnedBy": [],
      "archivedBy": [],
      "admins": [],
      "isLocked": false,
      "mutedBy": [],
      "createdAt": "2026-09-18T07:57:42.731Z",
      "updatedAt": "2026-09-18T07:58:07.711Z",
      "__v": 0,
      "latestMessage": {
        "_id": "6aacef0f5a7dcc4c6e188976",
        "sender": {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        },
        "content": "Hello Rahul",
        "chat": "6aaceef65a7dcc4c6e1888e8",
        "messageType": "text",
        "attachments": [],
        "isDeleted": false,
        "deletedFor": [],
        "isEdited": false,
        "isForwarded": true,
        "forwardedFrom": "6aaceedd5a7dcc4c6e18887d",
        "starredBy": [],
        "tags": [],
        "isBroadcast": false,
        "readBy": [],
        "deliveredTo": [],
        "reactions": [],
        "createdAt": "2026-09-18T07:58:07.050Z",
        "updatedAt": "2026-09-18T07:58:07.050Z",
        "__v": 0
      }
    },
    {
      "_id": "6aaceef85a7dcc4c6e1888f6",
      "chatName": "Office",
      "isGroupChat": true,
```

```
      "users": [
        {
          "_id": "6aaceedd5a7dcc4c6e18887d",
          "name": "Priya Verma",
          "mobile": "9000000002",
          "profilePicture": ""
        },
        {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "mobile": "9000000001",
          "profilePicture": ""
        }
      ],
      "groupAdmin": [
        {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        }
      ],
      "groupIcon": "",
      "groupDescription": "",
      "deletedBy": [],
      "pinnedBy": [],
      "archivedBy": [],
      "admins": [],
      "isLocked": false,
      "mutedBy": [],
      "createdAt": "2026-09-18T07:57:44.867Z",
      "updatedAt": "2026-09-18T07:57:45.853Z",
      "__v": 0,
      "latestMessage": {
        "_id": "6aaceef95a7dcc4c6e1888fb",
        "sender": {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        },
        "content": "Rahul Sharma created \"Office\" group",
        "chat": "6aaceef85a7dcc4c6e1888f6",
        "messageType": "text",
        "isDeleted": false,
        "deletedFor": [],
        "isEdited": false,
        "isForwarded": false,
        "starredBy": [],
        "tags": [],
        "isBroadcast": false,
        "attachments": [],
        "readBy": [],
        "deliveredTo": [],
        "reactions": [],
        "createdAt": "2026-09-18T07:57:45.596Z",
        "updatedAt": "2026-09-18T07:57:45.596Z",
        "__v": 0
      }
    }
  ],
  "pagination": {
    "currentPage": 1,
    "totalPages": 1,
    "totalGroups": 2,
    "hasMore": false,
    "limit": 50
  }
}
```

`GET /api/chats/unread` — Chats with unread messages

Access: Token + complete profile · **Body:** —

Only chats where `unseenCount > 0`, plus totals for badges.

Example

```
GET /api/chats/unread
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "totalUnseenChats": 1,
  "totalUnseenMessages": 1,
  "chats": [
    {
      "_id": "6aaceef65a7dcc4c6e1888e8",
      "chatName": "Weekend Trip",
      "isGroupChat": true,
      "users": [
        {
          "_id": "6aaceedd5a7dcc4c6e18887d",
          "name": "Priya Verma",
          "mobile": "9000000002",
          "profilePicture": ""
        },
        {
          "_id": "6aaceedf5a7dcc4c6e188884",
          "name": "Amit Kumar",
          "mobile": "9000000003",
          "profilePicture": ""
        },
        "…1 more"
      ],
      "groupAdmin": [
        {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        }
      ],
      "groupIcon": "",
      "groupDescription": "Plan for Sunday",
      "deletedBy": [],
      "pinnedBy": [],
      "archivedBy": [],
      "admins": [],
      "isLocked": false,
      "mutedBy": [],
      "createdAt": "2026-09-18T07:57:42.731Z",
      "updatedAt": "2026-09-18T07:58:07.711Z",
      "__v": 0,
      "latestMessage": {
        "_id": "6aacef0f5a7dcc4c6e188976",
        "sender": {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        },
        "content": "Hello Rahul",
        "chat": "6aaceef65a7dcc4c6e1888e8",
        "messageType": "text",
        "attachments": [],
        "isDeleted": false,
        "deletedFor": [],
        "isEdited": false,
        "isForwarded": true,
        "forwardedFrom": "6aaceedd5a7dcc4c6e18887d",
        "starredBy": [],
        "tags": [],
        "isBroadcast": false,
        "readBy": [],
        "deliveredTo": [],
        "reactions": [],
        "createdAt": "2026-09-18T07:58:07.050Z",
        "updatedAt": "2026-09-18T07:58:07.050Z",
        "__v": 0
      },
      "unseenCount": 1,
      "isPinned": false,
      "isMuted": false,
      "isArchived": false
```

```
    }
  ]
}
```

`POST /api/chats/group` — Create group

Access: Token + complete profile · **Body:** `multipart/form-data` (JSON also works when no file is sent)

Creates a group with you as admin, posts a "created group" system message and delivers it to members.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`name`	string	yes	Group name.
`users`	array or JSON string	yes	Members to add (you are added automatically). With multipart/FormData send `JSON.stringify([...ids])` ; with a JSON body a real array also works.
`description`	string	no	

Files

FIELD	TYPE	REQUIRED	DESCRIPTION
`groupIcon`	image	no	Multipart field name.

Example — JSON body — users is a STRING

```
POST /api/chats/group
Authorization: Bearer <token>
Content-Type: application/json

{
  "name": "Weekend Trip",
  "description": "Plan for Sunday",
  "users": "[\"6aaceedd5a7dcc4c6e18887d\",\"6aaceedf5a7dcc4c6e188884\"]"
}
```

Response `201`

```
{
  "success": true,
  "chat": {
    "_id": "6aaceef65a7dcc4c6e1888e8",
    "chatName": "Weekend Trip",
    "isGroupChat": true,
    "users": [
      {
        "privacySettings": {
          "lastSeen": "everyone",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": true,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": true
        },
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "district": null,
        "city": "Bhopal",
        "state": null,
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:17.878Z",
        "createdAt": "2026-09-18T07:57:17.881Z",
        "updatedAt": "2026-09-18T07:57:19.144Z",
        "__v": 0
      },
      {
        "privacySettings": {
          "lastSeen": "everyone",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": true,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": true
        },
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "mobile": "9000000003",
        "district": null,
        "city": "Indore",
        "state": null,
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:19.708Z",
        "createdAt": "2026-09-18T07:57:19.710Z",
        "updatedAt": "2026-09-18T07:57:21.011Z",
        "__v": 0
```

```
      },
      "…1 more"
    ],
    "groupAdmin": [
      {
        "privacySettings": {
          "lastSeen": "contacts",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": false,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": false
        },
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "mobile": "9000000001",
        "district": "Indore",
        "city": "Indore",
        "state": "Madhya Pradesh",
        "profilePicture": "",
        "about": "Busy right now",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:15.923Z",
        "createdAt": "2026-09-18T07:57:15.940Z",
        "updatedAt": "2026-09-18T07:57:40.250Z",
        "__v": 4,
        "inviteCode": "219dab9bd3e0"
      }
    ],
    "groupIcon": "",
    "groupDescription": "Plan for Sunday",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:42.731Z",
    "updatedAt": "2026-09-18T07:57:42.731Z",
    "__v": 0
  }
}
```


Example — JSON body — users as a real array (also accepted)

```
POST /api/chats/group
Authorization: Bearer <token>
Content-Type: application/json

{
  "name": "Office",
  "users": [
    "6aaceedd5a7dcc4c6e18887d"
  ]
}
```

Response `201`

```
{
  "success": true,
  "chat": {
    "_id": "6aaceef85a7dcc4c6e1888f6",
    "chatName": "Office",
    "isGroupChat": true,
    "users": [
      {
        "privacySettings": {
          "lastSeen": "everyone",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": true,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": true
        },
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "district": null,
        "city": "Bhopal",
        "state": null,
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:17.878Z",
        "createdAt": "2026-09-18T07:57:17.881Z",
        "updatedAt": "2026-09-18T07:57:19.144Z",
        "__v": 0
      },
      {
        "privacySettings": {
          "lastSeen": "contacts",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": false,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": false
        },
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "mobile": "9000000001",
        "district": "Indore",
        "city": "Indore",
        "state": "Madhya Pradesh",
        "profilePicture": "",
        "about": "Busy right now",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:15.923Z",
        "createdAt": "2026-09-18T07:57:15.940Z",
        "updatedAt": "2026-09-18T07:57:40.250Z",
        "__v": 4,
```

```
        "inviteCode": "219dab9bd3e0"
      }
    ],
    "groupAdmin": [
      {
        "privacySettings": {
          "lastSeen": "contacts",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": false,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": false
        },
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "mobile": "9000000001",
        "district": "Indore",
        "city": "Indore",
        "state": "Madhya Pradesh",
        "profilePicture": "",
        "about": "Busy right now",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:15.923Z",
        "createdAt": "2026-09-18T07:57:15.940Z",
        "updatedAt": "2026-09-18T07:57:40.250Z",
        "__v": 4,
        "inviteCode": "219dab9bd3e0"
      }
    ],
    "groupIcon": "",
    "groupDescription": "",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:44.867Z",
    "updatedAt": "2026-09-18T07:57:44.867Z",
    "__v": 0
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	Please provide group name and users	
400	users must be an array of user ids (or its JSON string)	
400	At least 2 users are required to create a group	
400	Image upload failed. Please check frontend FormData format.	

Real-time: `message-received` (system message) to each member.

`PUT /api/chats/group/update` — Update group

Access: Token + complete profile · **Body:** `multipart/form-data` (JSON also works when no file is sent)

Change name, description and/or icon. Group members only. Replacing the icon deletes the old file.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	
`chatName`	string	no	
`groupDescription`	string	no	

Files

FIELD	TYPE	REQUIRED	DESCRIPTION
`groupIcon`	image	no	

Example

```
PUT /api/chats/group/update
Authorization: Bearer <token>
Content-Type: application/json

{
  "chatId": "6aaceef65a7dcc4c6e1888e8",
  "chatName": "Weekend Trip 🚗",
  "groupDescription": "Sunday 8 AM"
}
```

Response `200`

```
{
  "success": true,
  "chat": {
    "_id": "6aaceef65a7dcc4c6e1888e8",
    "chatName": "Weekend Trip 🚗",
    "isGroupChat": true,
    "users": [
      {
        "privacySettings": {
          "lastSeen": "everyone",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": true,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": true
        },
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "district": null,
        "city": "Bhopal",
        "state": null,
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:17.878Z",
        "createdAt": "2026-09-18T07:57:17.881Z",
        "updatedAt": "2026-09-18T07:58:06.375Z",
        "__v": 1
      },
      {
        "privacySettings": {
          "lastSeen": "everyone",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": true,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": true
        },
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "mobile": "9000000003",
        "district": null,
        "city": "Indore",
        "state": null,
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:19.708Z",
        "createdAt": "2026-09-18T07:57:19.710Z",
        "updatedAt": "2026-09-18T07:57:21.011Z",
        "__v": 0
```

```
      },
      "…1 more"
    ],
    "groupAdmin": [
      {
        "privacySettings": {
          "lastSeen": "contacts",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": false,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": false
        },
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "mobile": "9000000001",
        "district": "Indore",
        "city": "Indore",
        "state": "Madhya Pradesh",
        "profilePicture": "",
        "about": "Busy right now",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:15.923Z",
        "createdAt": "2026-09-18T07:57:15.940Z",
        "updatedAt": "2026-09-18T07:57:40.250Z",
        "__v": 4,
        "inviteCode": "219dab9bd3e0"
      }
    ],
    "groupIcon": "",
    "groupDescription": "Sunday 8 AM",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:42.731Z",
    "updatedAt": "2026-09-18T07:58:19.237Z",
    "__v": 0,
    "latestMessage": "6aacef0f5a7dcc4c6e188976"
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	Chat ID is required	
404	Chat not found	
400	This is not a group chat	
403	You are not a member of this group	

Real-time: `group-updated` to the chat room.

`PUT /api/chats/group/rename` — Rename group

Access: Token + complete profile · **Body:** `application/json`

Name only; `update` above covers this too. Group members only.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	
`chatName`	string	yes	

Example

```
PUT /api/chats/group/rename
Authorization: Bearer <token>
Content-Type: application/json

{
  "chatId": "6aaceef65a7dcc4c6e1888e8",
  "chatName": "Sunday Trip"
}
```

Response `200`

```
{
  "success": true,
  "chat": {
    "_id": "6aaceef65a7dcc4c6e1888e8",
    "chatName": "Sunday Trip",
    "isGroupChat": true,
    "users": [
      {
        "privacySettings": {
          "lastSeen": "everyone",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": true,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": true
        },
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "district": null,
        "city": "Bhopal",
        "state": null,
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:17.878Z",
        "createdAt": "2026-09-18T07:57:17.881Z",
        "updatedAt": "2026-09-18T07:58:06.375Z",
        "__v": 1
      },
      {
        "privacySettings": {
          "lastSeen": "everyone",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": true,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": true
        },
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "mobile": "9000000003",
        "district": null,
        "city": "Indore",
        "state": null,
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:19.708Z",
        "createdAt": "2026-09-18T07:57:19.710Z",
        "updatedAt": "2026-09-18T07:57:21.011Z",
        "__v": 0
```

```
      },
      "…1 more"
    ],
    "groupAdmin": [
      {
        "privacySettings": {
          "lastSeen": "contacts",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": false,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": false
        },
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "mobile": "9000000001",
        "district": "Indore",
        "city": "Indore",
        "state": "Madhya Pradesh",
        "profilePicture": "",
        "about": "Busy right now",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:15.923Z",
        "createdAt": "2026-09-18T07:57:15.940Z",
        "updatedAt": "2026-09-18T07:57:40.250Z",
        "__v": 4,
        "inviteCode": "219dab9bd3e0"
      }
    ],
    "groupIcon": "",
    "groupDescription": "Sunday 8 AM",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:42.731Z",
    "updatedAt": "2026-09-18T07:58:19.931Z",
    "__v": 0,
    "latestMessage": "6aacef0f5a7dcc4c6e188976"
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	chatName is required	
404	Chat not found	
400	This is not a group chat	
403	You are not a member of this group	

`PUT /api/chats/group/add` — Add members

Access: Token + complete profile · **Body:** `application/json`

Adds users to the group. Any group member can add. Adding someone already in the group has no effect.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	
`userIds`	array	yes	A JSON array (its JSON string also works).

Example — userIds is a real ARRAY

```
PUT /api/chats/group/add
Authorization: Bearer <token>
Content-Type: application/json

{
  "chatId": "6aaceef65a7dcc4c6e1888e8",
  "userIds": [
    "6aaceee15a7dcc4c6e18888b"
  ]
}
```

Response `200`

```
{
  "success": true,
  "chat": {
    "_id": "6aaceef65a7dcc4c6e1888e8",
    "chatName": "Sunday Trip",
    "isGroupChat": true,
    "users": [
      {
        "privacySettings": {
          "lastSeen": "everyone",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": true,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": true
        },
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "district": null,
        "city": "Bhopal",
        "state": null,
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:17.878Z",
        "createdAt": "2026-09-18T07:57:17.881Z",
        "updatedAt": "2026-09-18T07:58:06.375Z",
        "__v": 1
      },
      {
        "privacySettings": {
          "lastSeen": "everyone",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": true,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": true
        },
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "mobile": "9000000003",
        "district": null,
        "city": "Indore",
        "state": null,
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:19.708Z",
        "createdAt": "2026-09-18T07:57:19.710Z",
        "updatedAt": "2026-09-18T07:57:21.011Z",
        "__v": 0
```

```
      },
      "…2 more"
    ],
    "groupAdmin": [
      {
        "privacySettings": {
          "lastSeen": "contacts",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": false,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": false
        },
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "mobile": "9000000001",
        "district": "Indore",
        "city": "Indore",
        "state": "Madhya Pradesh",
        "profilePicture": "",
        "about": "Busy right now",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:15.923Z",
        "createdAt": "2026-09-18T07:57:15.940Z",
        "updatedAt": "2026-09-18T07:57:40.250Z",
        "__v": 4,
        "inviteCode": "219dab9bd3e0"
      }
    ],
    "groupIcon": "",
    "groupDescription": "Sunday 8 AM",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:42.731Z",
    "updatedAt": "2026-09-18T07:58:20.622Z",
    "__v": 0,
    "latestMessage": "6aacef0f5a7dcc4c6e188976"
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	userIds must be a non-empty array	
404	Chat not found	
400	This is not a group chat	
403	You are not a member of this group	

Notes

- After success, emit socket `user-added-to-group` so the new member and others update live.

`PUT /api/chats/group/remove` — Remove member

Access: Token + complete profile · **Body:** `application/json`

Admins can remove anyone except the group owner; a member can remove themselves. Removing also drops their admin role.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	
`userId`	ObjectId	yes	

Example

```
PUT /api/chats/group/remove
Authorization: Bearer <token>
Content-Type: application/json

{
  "chatId": "6aaceef65a7dcc4c6e1888e8",
  "userId": "6aaceee15a7dcc4c6e18888b"
}
```

Response `200`

```
{
  "success": true,
  "chat": {
    "_id": "6aaceef65a7dcc4c6e1888e8",
    "chatName": "Sunday Trip",
    "isGroupChat": true,
    "users": [
      {
        "privacySettings": {
          "lastSeen": "everyone",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": true,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": true
        },
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "district": null,
        "city": "Bhopal",
        "state": null,
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:17.878Z",
        "createdAt": "2026-09-18T07:57:17.881Z",
        "updatedAt": "2026-09-18T07:58:06.375Z",
        "__v": 1
      },
      {
        "privacySettings": {
          "lastSeen": "everyone",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": true,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": true
        },
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "mobile": "9000000003",
        "district": null,
        "city": "Indore",
        "state": null,
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:19.708Z",
        "createdAt": "2026-09-18T07:57:19.710Z",
        "updatedAt": "2026-09-18T07:57:21.011Z",
        "__v": 0
```

```
      },
      "…1 more"
    ],
    "groupAdmin": [
      {
        "privacySettings": {
          "lastSeen": "contacts",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": false,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": false
        },
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "mobile": "9000000001",
        "district": "Indore",
        "city": "Indore",
        "state": "Madhya Pradesh",
        "profilePicture": "",
        "about": "Busy right now",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:15.923Z",
        "createdAt": "2026-09-18T07:57:15.940Z",
        "updatedAt": "2026-09-18T07:57:40.250Z",
        "__v": 4,
        "inviteCode": "219dab9bd3e0"
      }
    ],
    "groupIcon": "",
    "groupDescription": "Sunday 8 AM",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:42.731Z",
    "updatedAt": "2026-09-18T07:58:27.638Z",
    "__v": 2,
    "latestMessage": "6aacef205a7dcc4c6e1889d7"
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
403	Only group admins can remove members	Non-admin removing someone else
400	userId is required	
404	Chat not found	
400	This is not a group chat	
403	removingSelf	

Notes

- The owner cannot be removed — they must use `PUT /api/chats/group/leave`.

- After success, emit socket `user-removed-from-group`.

PUT /api/chats/group/leave — Leave group

Access: Token + complete profile · **Body:** `application/json`

You leave the group. If you were an admin and no admin remains, the next member becomes admin. If you were the last member, the group is deleted (`groupDeleted: true`).

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Example — Admin leaves — admin role passes to the next member

```
PUT /api/chats/group/leave
Authorization: Bearer <token>
Content-Type: application/json

{
  "chatId": "6aaceef85a7dcc4c6e1888f6"
}
```

Response `200`

```
{
  "success": true,
  "message": "You have left the group",
  "chat": {
    "_id": "6aaceef85a7dcc4c6e1888f6",
    "chatName": "Office",
    "isGroupChat": true,
    "users": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "profilePicture": ""
      }
    ],
    "groupAdmin": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "profilePicture": ""
      }
    ],
    "groupIcon": "",
    "groupDescription": "",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:44.867Z",
    "updatedAt": "2026-09-18T07:59:08.994Z",
    "__v": 1,
    "latestMessage": "6aaceef95a7dcc4c6e1888fb"
  }
}
```

Example

```
PUT /api/chats/group/leave
Authorization: Bearer <token>
Content-Type: application/json

{
  "chatId": "6aaceef65a7dcc4c6e1888e8"
}
```

Response `200`


```
{
  "success": true,
  "message": "You have left the group",
  "chat": {
    "_id": "6aaceef65a7dcc4c6e1888e8",
    "chatName": "Sunday Trip",
    "isGroupChat": true,
    "users": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "profilePicture": ""
      },
      {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "mobile": "9000000001",
        "profilePicture": ""
      }
    ],
    "groupAdmin": [
      {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      }
    ],
    "groupIcon": "",
    "groupDescription": "Sunday 8 AM",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:42.731Z",
    "updatedAt": "2026-09-18T07:58:28.448Z",
    "__v": 3,
    "latestMessage": "6aacef205a7dcc4c6e1889d7"
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	Chat ID is required	
404	Group not found	
400	This is not a group chat	
400	You are not a member of this group	
500	Server error	

`PUT /api/chats/group/add-admin` — Make admin

Access: Token + complete profile · **Body:** `application/json`

Promotes a member to admin. Only the group owner (`groupAdmin`) can do this.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	
`userId`	ObjectId	yes	Must already be a member.

Example

```
PUT /api/chats/group/add-admin
Authorization: Bearer <token>
Content-Type: application/json

{
  "chatId": "6aaceef65a7dcc4c6e1888e8",
  "userId": "6aaceedd5a7dcc4c6e18887d"
}
```

Response `200`

```
{
  "success": true,
  "message": "Admin added successfully",
  "chat": {
    "_id": "6aaceef65a7dcc4c6e1888e8",
    "chatName": "Sunday Trip",
    "isGroupChat": true,
    "users": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "profilePicture": ""
      },
      {
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "profilePicture": ""
      },
      "…2 more"
    ],
    "groupAdmin": [
      {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      }
    ],
    "groupIcon": "",
    "groupDescription": "Sunday 8 AM",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "profilePicture": ""
      }
    ],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:42.731Z",
    "updatedAt": "2026-09-18T07:58:21.314Z",
    "__v": 1,
    "latestMessage": "6aacef0f5a7dcc4c6e188976"
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	Chat ID and User ID are required	
404	Group not found	
400	This is not a group chat	
403	Only group owners can add other admins	
400	You are already the group owner (super admin). No need to add yourself as admin.	
400	User is not a member of this group	
400	User is already an admin	
500	Server error	

`PUT /api/chats/group/remove-admin` — Remove admin

Access: Token + complete profile · **Body:** `application/json`

Demotes an admin. The owner cannot be removed.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	
`userId`	ObjectId	yes	

Example

```
PUT /api/chats/group/remove-admin
Authorization: Bearer <token>
Content-Type: application/json

{
  "chatId": "6aaceef65a7dcc4c6e1888e8",
  "userId": "6aaceedd5a7dcc4c6e18887d"
}
```

Response `200`

```
{
  "success": true,
  "message": "Admin removed successfully",
  "chat": {
    "_id": "6aaceef65a7dcc4c6e1888e8",
    "chatName": "Sunday Trip",
    "isGroupChat": true,
    "users": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "profilePicture": ""
      },
      {
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "profilePicture": ""
      },
      "…2 more"
    ],
    "groupAdmin": [
      {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      }
    ],
    "groupIcon": "",
    "groupDescription": "Sunday 8 AM",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:42.731Z",
    "updatedAt": "2026-09-18T07:58:22.220Z",
    "__v": 2,
    "latestMessage": "6aacef0f5a7dcc4c6e188976"
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	Chat ID and User ID are required	
404	Group not found	
400	This is not a group chat	
403	Only the group owner can remove admins	
400	Cannot remove the main group owner (super admin)	
400	User is not an admin in this group	
500	Server error	

PUT /api/chats/group/lock — Lock group

Access: Token + complete profile · **Body:** `application/json`

Only admins can send messages while locked. Admin only.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Example

```
PUT /api/chats/group/lock
Authorization: Bearer <token>
Content-Type: application/json

{
  "chatId": "6aaceef65a7dcc4c6e1888e8"
}
```

Response `200`


```
{
  "success": true,
  "message": "Chat locked. Only admins can send messages.",
  "chat": {
    "_id": "6aaceef65a7dcc4c6e1888e8",
    "chatName": "Sunday Trip",
    "isGroupChat": true,
    "users": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "profilePicture": ""
      },
      {
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "profilePicture": ""
      },
      "…2 more"
    ],
    "groupAdmin": [
      {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      }
    ],
    "groupIcon": "",
    "groupDescription": "Sunday 8 AM",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": true,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:42.731Z",
    "updatedAt": "2026-09-18T07:58:23.122Z",
    "__v": 2,
    "latestMessage": "6aacef0f5a7dcc4c6e188976"
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	Chat ID is required	
404	Group not found	
400	This is not a group chat	
403	Only admins can lock the chat	

STATUS	MESSAGE	WHEN
500	Server error	

Notes

- Non-admins sending to a locked group get 403 `Chat is locked. Only admins can send messages.`

PUT /api/chats/group/unlock — Unlock group

Access: Token + complete profile · **Body:** `application/json`

Everyone can send again. Admin only.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Example

```
PUT /api/chats/group/unlock
Authorization: Bearer <token>
Content-Type: application/json

{
  "chatId": "6aaceef65a7dcc4c6e1888e8"
}
```

Response `200`

```
{
  "success": true,
  "message": "Chat unlocked. All members can send messages.",
  "chat": {
    "_id": "6aaceef65a7dcc4c6e1888e8",
    "chatName": "Sunday Trip",
    "isGroupChat": true,
    "users": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "profilePicture": ""
      },
      {
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "profilePicture": ""
      },
      "…2 more"
    ],
    "groupAdmin": [
      {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      }
    ],
    "groupIcon": "",
    "groupDescription": "Sunday 8 AM",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:42.731Z",
    "updatedAt": "2026-09-18T07:58:25.606Z",
    "__v": 2,
    "latestMessage": "6aacef205a7dcc4c6e1889d7"
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	Chat ID is required	
404	Group not found	
400	This is not a group chat	
403	Only admins can unlock the chat	
500	Server error	

`DELETE /api/chats/group/:chatId` — Delete group

Access: Token + complete profile · **Body:** —

Deletes the group, all its messages and their files. Group owner only.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Example

```
DELETE /api/chats/group/6aaceef65a7dcc4c6e1888e8
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Group deleted successfully"
}
```

Error responses

STATUS	MESSAGE	WHEN
403	Only group admin can delete the group	Non-admin
404	Group not found	
400	This is not a group chat	
500	Server error	

`GET /api/chats/group/:chatId/info` — Group info

Access: Token + complete profile · **Body:** —

Members, admins, your admin status and the shared media (images, videos, documents, audio).
Members only.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Example

```
GET /api/chats/group/6aaceef65a7dcc4c6e1888e8/info
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "group": {
    "_id": "6aaceef65a7dcc4c6e1888e8",
    "chatName": "Sunday Trip",
    "groupIcon": "",
    "groupDescription": "Sunday 8 AM",
    "createdAt": "2026-09-18T07:57:42.731Z",
    "members": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook"
      },
      {
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "mobile": "9000000003",
        "profilePicture": "",
        "about": "Hey there! I am using TeleBook"
      },
      "…2 more"
    ],
    "admins": [],
    "totalMembers": 4,
    "totalAdmins": 1,
    "isAdmin": true,
    "groupAdmin": [
      {
        "privacySettings": {
          "lastSeen": "contacts",
          "profilePhoto": "everyone",
          "about": "everyone",
          "status": "contacts"
        },
        "notificationSettings": {
          "messageNotifications": true,
          "groupNotifications": false,
          "statusNotifications": true,
          "channelNotifications": true,
          "sound": true,
          "vibration": false
        },
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "mobile": "9000000001",
        "district": "Indore",
        "city": "Indore",
        "state": "Madhya Pradesh",
        "profilePicture": "",
        "about": "Busy right now",
        "isOnline": false,
        "isVerified": true,
        "isDeleted": false,
        "blockedUsers": [],
        "isdCode": "91",
        "lastSeen": "2026-09-18T07:57:15.923Z",
        "createdAt": "2026-09-18T07:57:15.940Z",
        "updatedAt": "2026-09-18T07:57:40.250Z",
        "__v": 4,
        "inviteCode": "219dab9bd3e0"
      }
    ],
    "totalMessages": 4
  },
  "media": {
    "images": {
      "count": 0,
      "items": []
    },
```

```
    "videos": {
      "count": 0,
      "items": []
    },
    "documents": {
      "count": 0,
      "items": []
    },
    "audios": {
      "count": 0,
      "items": []
    },
    "total": 0
  }
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Group not found	
400	This is not a group chat	
403	You are not a member of this group	

`PUT /api/chats/:chatId/mute` — Mute chat

Access: Token + complete profile · **Body:** `application/json`

Mutes for you only: no push notifications for this chat until `mutedUntil`. Unread counts still increase.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`duration`	number	yes	Hours: `8`, `24`, `168`, … or `0` for forever. A numeric string such as `"8"` is also accepted.

Example — 8 hours

```
PUT /api/chats/6aaceef55a7dcc4c6e1888e2/mute
Authorization: Bearer <token>
Content-Type: application/json

{
  "duration": 8
}
```

Response `200`

```
{
  "success": true,
  "message": "Chat muted",
  "mutedUntil": "2026-09-18T15:58:29.505Z"
}
```

Example — Forever

```
PUT /api/chats/6aaceef55a7dcc4c6e1888e2/mute
Authorization: Bearer <token>
Content-Type: application/json

{
  "duration": 0
}
```

Response `200`

```
{
  "success": true,
  "message": "Chat muted",
  "mutedUntil": "2126-08-25T07:58:29.972Z"
}
```

Error responses

STATUS	MESSAGE	WHEN
400	duration must be a number of hours (0 = forever)	Missing duration
404	Chat not found	
403	You are not part of this chat	

`PUT /api/chats/:chatId/unmute` — Unmute chat

Access: Token + complete profile · **Body:** `application/json`

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Example

```
PUT /api/chats/6aaceef55a7dcc4c6e1888e2/unmute
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Chat unmuted"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Chat not found	

PUT /api/chats/:chatId/pin — Pin chat

Access: Token + complete profile · **Body:** `application/json`

Pinned chats sort to the top of `GET /api/chats` for you. Participants only.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Example

```
PUT /api/chats/6aaceef55a7dcc4c6e1888e2/pin
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Chat pinned"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Chat not found	
403	You are not part of this chat	

PUT /api/chats/:chatId/unpin — Unpin chat

Access: Token + complete profile · **Body:** `application/json`

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Example

```
PUT /api/chats/6aaceef55a7dcc4c6e1888e2/unpin
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Chat unpinned"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Chat not found	

PUT /api/chats/:chatId/archive — Archive chat

Access: Token + complete profile · **Body:** `application/json`

Sets `isArchived: true` for you. The chat still appears in `GET /api/chats`. Participants only.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Example

```
PUT /api/chats/6aaceef55a7dcc4c6e1888e2/archive
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Chat archived"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Chat not found	
403	You are not part of this chat	

PUT /api/chats/:chatId/unarchive — Unarchive chat

Access: Token + complete profile · **Body:** `application/json`

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Example

```
PUT /api/chats/6aaceef55a7dcc4c6e1888e2/unarchive
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Chat unarchived"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Chat not found	

`GET /api/chats/:chatId/info` — Chat info (1-to-1)

Access: Token + complete profile · **Body:** —

The chat plus shared media grouped by type. Participants only.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Example

```
GET /api/chats/6aaceef55a7dcc4c6e1888e2/info
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "isGroupChat": false,
  "chat": {
    "_id": "6aaceef55a7dcc4c6e1888e2",
    "createdAt": "2026-09-18T07:57:41.363Z",
    "otherUser": {
      "_id": "6aaceedd5a7dcc4c6e18887d",
      "name": "Priya Verma",
      "mobile": "9000000002",
      "profilePicture": "",
      "about": "Hey there! I am using TeleBook",
      "isOnline": false,
      "lastSeen": "2026-09-18T07:57:17.878Z"
    },
    "totalMessages": 6,
    "starredMessages": 1
  },
  "media": {
    "images": {
      "count": 1,
      "items": [
        {
          "_id": "6aaceefe5a7dcc4c6e18891a",
          "sender": {
            "_id": "6aaceedb5a7dcc4c6e188876",
            "name": "Rahul Sharma",
            "profilePicture": ""
          },
          "messageType": "image",
          "attachments": [
            {
              "url": "uploads/messages/images/media-1789718248836-13397368.png",
              "filename": "photo.png",
              "fileType": "image/png",
              "fileSize": 70,
              "_id": "6aaceefe5a7dcc4c6e18891b"
            }
          ],
          "createdAt": "2026-09-18T07:57:50.943Z"
        }
      ]
    },
    "videos": {
      "count": 0,
      "items": []
    },
    "documents": {
      "count": 0,
      "items": []
    },
    "audios": {
      "count": 0,
      "items": []
    },
    "total": 1
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
404	Chat not found	
403	You are not a participant of this chat	

`DELETE /api/chats/:chatId` — Delete chat (for me)

Access: Token + complete profile · **Body:** —

Hides the chat and its messages for you. It reappears if a new message arrives (older messages stay hidden). When every participant has deleted it, the chat and its files are removed permanently.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Example

```
DELETE /api/chats/6aaceef55a7dcc4c6e1888e2
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Chat deleted for you"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Chat not found	
403	You are not part of this chat	
500	Server error	

`DELETE /api/chats/:chatId/clear` — Clear chat

Access: Token + complete profile · **Body:** —

Hides all current messages for you. The chat stays in your list. Others are unaffected.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Example

```
DELETE /api/chats/6aaceef55a7dcc4c6e1888e2/clear
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Chat cleared successfully",
  "clearedCount": 7
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Chat not found	
403	You are not part of this chat	
500	Server error	

6.4 Messages

Sending, reading, searching, forwarding, deleting, starring, reacting to and tagging messages.

`POST /api/messages` — Send message

Access: Token · **Body:** `application/json`

Saves the message and delivers it: every other member gets socket `message-received`; members with no live socket get a push unless they muted the chat or turned notifications off. Updates chat lists and unread counts. You must be a member of the chat.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	
`content`	string	no	Text. Empty for media-only.
`messageType`	string	no	`text` (default) | `emoji` | `image` | `video` | `audio` | `document`.
`replyTo`	ObjectId	no	Message being replied to; returned populated.
`attachments`	array	no	Objects returned by `POST /api/users/media`: `{ url, filename, fileType, fileSize }`.

Example — Text

```
POST /api/messages
Authorization: Bearer <token>
Content-Type: application/json

{
  "chatId": "6aaceef55a7dcc4c6e1888e2",
  "content": "Hi Priya!",
  "messageType": "text"
}
```

Response `201`


```
{
  "success": true,
  "message": {
    "sender": {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "profilePicture": ""
    },
    "content": "Hi Priya!",
    "chat": {
      "_id": "6aaceef55a7dcc4c6e1888e2",
      "chatName": "sender",
      "isGroupChat": false,
      "users": [
        {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        },
        {
          "_id": "6aaceedd5a7dcc4c6e18887d",
          "name": "Priya Verma",
          "profilePicture": ""
        }
      ],
      "groupAdmin": [],
      "groupIcon": "",
      "groupDescription": "",
      "deletedBy": [],
      "pinnedBy": [],
      "archivedBy": [],
      "admins": [],
      "isLocked": false,
      "mutedBy": [],
      "createdAt": "2026-09-18T07:57:41.363Z",
      "updatedAt": "2026-09-18T07:57:41.363Z",
      "__v": 0
    },
    "messageType": "text",
    "attachments": [],
    "isDeleted": false,
    "deletedFor": [],
    "isEdited": false,
    "isForwarded": false,
    "starredBy": [],
    "tags": [],
    "isBroadcast": false,
    "_id": "6aaceefb5a7dcc4c6e188906",
    "readBy": [],
    "deliveredTo": [],
    "reactions": [],
    "createdAt": "2026-09-18T07:57:47.295Z",
    "updatedAt": "2026-09-18T07:57:47.295Z",
    "__v": 0
  }
}
```

Example — Reply

```
POST /api/messages
Authorization: Bearer <token>
Content-Type: application/json

{
  "chatId": "6aaceef55a7dcc4c6e1888e2",
  "content": "Hello Rahul",
  "messageType": "text",
  "replyTo": "6aaceefb5a7dcc4c6e188906"
}
```

Response `201`

```
{
  "success": true,
  "message": {
    "sender": {
      "_id": "6aaceedd5a7dcc4c6e18887d",
      "name": "Priya Verma",
      "profilePicture": ""
    },
    "content": "Hello Rahul",
    "chat": {
      "_id": "6aaceef55a7dcc4c6e1888e2",
      "chatName": "sender",
      "isGroupChat": false,
      "users": [
        {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        },
        {
          "_id": "6aaceedd5a7dcc4c6e18887d",
          "name": "Priya Verma",
          "profilePicture": ""
        }
      ],
      "groupAdmin": [],
      "groupIcon": "",
      "groupDescription": "",
      "deletedBy": [],
      "pinnedBy": [],
      "archivedBy": [],
      "admins": [],
      "isLocked": false,
      "mutedBy": [],
      "createdAt": "2026-09-18T07:57:41.363Z",
      "updatedAt": "2026-09-18T07:57:48.203Z",
      "__v": 0,
      "latestMessage": "6aaceefb5a7dcc4c6e188906"
    },
    "messageType": "text",
    "attachments": [],
    "isDeleted": false,
    "deletedFor": [],
    "replyTo": {
      "_id": "6aaceefb5a7dcc4c6e188906",
      "sender": {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      },
      "content": "Hi Priya!",
      "messageType": "text",
      "attachments": []
    },
    "isEdited": false,
    "isForwarded": false,
    "starredBy": [],
    "tags": [],
    "isBroadcast": false,
    "_id": "6aaceefc5a7dcc4c6e18890f",
    "readBy": [],
    "deliveredTo": [],
    "reactions": [],
    "createdAt": "2026-09-18T07:57:48.925Z",
    "updatedAt": "2026-09-18T07:57:48.925Z",
    "__v": 0
  }
}
```


Example — With attachment (upload first via /api/users/media)

```
POST /api/messages
Authorization: Bearer <token>
Content-Type: application/json

{
  "chatId": "6aaceef55a7dcc4c6e1888e2",
  "content": "",
  "messageType": "image",
  "attachments": [
    {
      "url": "uploads/messages/images/media-1789718248836-13397368.png",
      "filename": "photo.png",
      "fileType": "image/png",
      "fileSize": 70
    }
  ]
}
```

Response `201`

```
{
  "success": true,
  "message": {
    "sender": {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "profilePicture": ""
    },
    "content": "",
    "chat": {
      "_id": "6aaceef55a7dcc4c6e1888e2",
      "chatName": "sender",
      "isGroupChat": false,
      "users": [
        {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        },
        {
          "_id": "6aaceedd5a7dcc4c6e18887d",
          "name": "Priya Verma",
          "profilePicture": ""
        }
      ],
      "groupAdmin": [],
      "groupIcon": "",
      "groupDescription": "",
      "deletedBy": [],
      "pinnedBy": [],
      "archivedBy": [],
      "admins": [],
      "isLocked": false,
      "mutedBy": [],
      "createdAt": "2026-09-18T07:57:41.363Z",
      "updatedAt": "2026-09-18T07:57:50.259Z",
      "__v": 0,
      "latestMessage": "6aaceefc5a7dcc4c6e18890f"
    },
    "messageType": "image",
    "attachments": [
      {
        "url": "uploads/messages/images/media-1789718248836-13397368.png",
        "filename": "photo.png",
        "fileType": "image/png",
        "fileSize": 70,
        "_id": "6aaceefe5a7dcc4c6e18891b"
      }
    ],
    "isDeleted": false,
    "deletedFor": [],
    "isEdited": false,
    "isForwarded": false,
    "starredBy": [],
    "tags": [],
    "isBroadcast": false,
    "_id": "6aaceefe5a7dcc4c6e18891a",
    "readBy": [],
    "deliveredTo": [],
    "reactions": [],
    "createdAt": "2026-09-18T07:57:50.943Z",
    "updatedAt": "2026-09-18T07:57:50.943Z",
    "__v": 0
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
403	You have blocked this contact. Unblock to send messages.	You blocked this contact
403	Chat is locked. Only admins can send messages.	Group locked — non-admin is refused
400	Chat ID is required	
404	Chat not found	
403	This account has been deleted	

Notes

- Do **not** also emit socket `new-message` — the server has already delivered it.
- Upload the file first via `POST /api/users/media`, then send its result in `attachments`.
- **Blocking (1-to-1):** if the other person blocked you, the message is saved and returned to you normally but is never delivered to them (single tick forever). If you blocked them, sending fails with 403.
- Sending to a deleted account returns 403 `This account has been deleted`.

Real-time: `message-received` to each recipient.

POST /api/messages/forward — Forward messages

Access: Token + complete profile · **Body:** `application/json`

Copies each message into each target chat, in order, marked `isForwarded` with `forwardedFrom` = original sender. Each copy is delivered like a new message. You must be a member of every source and target chat.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageIds`	array	yes	
`chatIds`	array	yes	Target chats.

Example

```
POST /api/messages/forward
Authorization: Bearer <token>
Content-Type: application/json

{
  "messageIds": [
    "6aaceefc5a7dcc4c6e18890f"
  ],
  "chatIds": [
    "6aaceef65a7dcc4c6e1888e8"
  ]
}
```

Response `201`

```
{
  "success": true,
  "totalForwarded": 1,
  "messages": [
    {
      "_id": "6aacef0f5a7dcc4c6e188976",
      "sender": {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      },
      "content": "Hello Rahul",
      "chat": "6aaceef65a7dcc4c6e1888e8",
      "messageType": "text",
      "attachments": [],
      "isDeleted": false,
      "deletedFor": [],
      "isEdited": false,
      "isForwarded": true,
      "forwardedFrom": {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "profilePicture": ""
      },
      "starredBy": [],
      "tags": [],
      "isBroadcast": false,
      "readBy": [],
      "deliveredTo": [],
      "reactions": [],
      "createdAt": "2026-09-18T07:58:07.050Z",
      "updatedAt": "2026-09-18T07:58:07.050Z",
      "__v": 0
    }
  ]
}
```


Error responses

STATUS	MESSAGE	WHEN
400	At least one message ID is required	
400	At least one chat ID is required	
404	No messages found	
403	You are not part of one or more of these chats	

Real-time: `message-received` to members of each target chat.

`GET /api/messages/starred` — Starred messages

Access: Token + complete profile · **Body:** —

Every message you have starred, across chats.

Example

```
GET /api/messages/starred
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "messages": [
    {
      "_id": "6aaceefc5a7dcc4c6e18890f",
      "sender": {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "profilePicture": ""
      },
      "content": "Hello Rahul",
      "chat": {
        "_id": "6aaceef55a7dcc4c6e1888e2",
        "chatName": "sender",
        "isGroupChat": false,
        "users": [
          "6aaceedb5a7dcc4c6e188876",
          "6aaceedd5a7dcc4c6e18887d"
        ],
        "groupAdmin": [],
        "groupIcon": "",
        "groupDescription": "",
        "deletedBy": [],
        "pinnedBy": [],
        "archivedBy": [],
        "admins": [],
        "isLocked": false,
        "mutedBy": [],
        "createdAt": "2026-09-18T07:57:41.363Z",
        "updatedAt": "2026-09-18T07:57:51.829Z",
        "__v": 0,
        "latestMessage": "6aaceefe5a7dcc4c6e18891a"
      },
      "messageType": "text",
      "attachments": [],
      "isDeleted": false,
      "deletedFor": [],
      "replyTo": "6aaceefb5a7dcc4c6e188906",
      "isEdited": false,
      "isForwarded": false,
      "starredBy": [
        "6aaceedb5a7dcc4c6e188876"
      ],
      "tags": [],
      "isBroadcast": false,
      "readBy": [],
      "deliveredTo": [],
      "reactions": [],
      "createdAt": "2026-09-18T07:57:48.925Z",
      "updatedAt": "2026-09-18T07:57:58.942Z",
      "__v": 1
    }
  ]
}
```

`DELETE /api/messages/delete-for-me` — Delete for me (many)

Access: Token · **Body:** `application/json`

Hides several messages for you only.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageIds`	array	yes	

Example

```
DELETE /api/messages/delete-for-me
Authorization: Bearer <token>
Content-Type: application/json

{
  "messageIds": [
    "6aacef125a7dcc4c6e188989"
  ]
}
```

Response `200`

```
{
  "success": true,
  "message": "Messages deleted for you"
}
```

Error responses

STATUS	MESSAGE	WHEN
400	Please provide an array of message IDs	
500	Server error	

`DELETE /api/messages` — Delete for everyone

Access: Token + complete profile · **Body:** `application/json`

Deletes your own messages for everyone: content becomes `"This message was deleted"`, attachments are removed from storage. Messages sent by others are skipped and listed in `skippedMessageIds`.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageIds`	array	yes	

Example — For everyone (body)

```
DELETE /api/messages
Authorization: Bearer <token>
Content-Type: application/json

{
  "messageIds": [
    "6aacef135a7dcc4c6e188992"
  ]
}
```

Response `200`

```
{
  "success": true,
  "message": "1 message(s) deleted successfully",
  "deletedCount": 1,
  "deletedMessageIds": [
    "6aacef135a7dcc4c6e188992"
  ],
  "skippedCount": 0,
  "skippedMessageIds": []
}
```

Error responses

STATUS	MESSAGE	WHEN
400	messageIds array is required	
404	No messages found	
403	You can only delete your own messages	
500	Server error	

Real-time: `messages-deleted` to the chat room.

`GET /api/messages/:chatId` — Get messages (and mark as read)

Access: Token + complete profile · **Body:** —

Paginated messages, **newest first** — reverse for display. Opening a chat marks every unread message in it as read and delivered, tells each sender (`message-read-update`) and resets the chat's unread count. Members only.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`chatId`	ObjectId	yes	

Query parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`page`	number	no	Default 1.
`limit`	number	no	Default 50.
`search`	string	no	Search mode: case-insensitive text match. Returns the matches plus 5 messages of context on each side, **oldest first**, with `matchedIds` to highlight.

Example — Search

```
GET /api/messages/6aaceef55a7dcc4c6e1888e2?search=hello
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "isSearchResult": true,
  "messages": [
    {
      "_id": "6aaceefb5a7dcc4c6e188906",
      "sender": {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      },
      "content": "Hi Priya!",
      "chat": {
        "_id": "6aaceef55a7dcc4c6e1888e2",
        "chatName": "sender",
        "isGroupChat": false,
        "users": [
          "6aaceedb5a7dcc4c6e188876",
          "6aaceedd5a7dcc4c6e18887d"
        ],
        "groupAdmin": [],
        "groupIcon": "",
        "groupDescription": "",
        "deletedBy": [],
        "pinnedBy": [],
        "archivedBy": [],
        "admins": [],
        "isLocked": false,
        "mutedBy": [],
        "createdAt": "2026-09-18T07:57:41.363Z",
        "updatedAt": "2026-09-18T07:57:51.829Z",
        "__v": 0,
        "latestMessage": "6aaceefe5a7dcc4c6e18891a"
      },
      "messageType": "text",
      "attachments": [],
      "isDeleted": false,
      "deletedFor": [],
      "isEdited": false,
      "isForwarded": false,
      "starredBy": [],
      "tags": [],
      "isBroadcast": false,
      "readBy": [
        {
          "user": "6aaceedd5a7dcc4c6e18887d",
          "readAt": "2026-09-18T07:57:54.953Z",
          "_id": "6aacef025a7dcc4c6e188934"
        }
      ],
      "deliveredTo": [
        {
          "user": "6aaceedd5a7dcc4c6e18887d",
          "deliveredAt": "2026-09-18T07:57:54.953Z",
          "_id": "6aacef035a7dcc4c6e188936"
        }
      ],
      "reactions": [],
      "createdAt": "2026-09-18T07:57:47.295Z",
      "updatedAt": "2026-09-18T07:57:55.192Z",
      "__v": 0
    },
    {
      "_id": "6aaceefc5a7dcc4c6e18890f",
      "sender": {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "profilePicture": ""
      },
      "content": "Hello Rahul",
      "chat": {
        "_id": "6aaceef55a7dcc4c6e1888e2",
```

```
        "chatName": "sender",
        "isGroupChat": false,
        "users": [
          "6aaceedb5a7dcc4c6e188876",
          "6aaceedd5a7dcc4c6e18887d"
        ],
        "groupAdmin": [],
        "groupIcon": "",
        "groupDescription": "",
        "deletedBy": [],
        "pinnedBy": [],
        "archivedBy": [],
        "admins": [],
        "isLocked": false,
        "mutedBy": [],
        "createdAt": "2026-09-18T07:57:41.363Z",
        "updatedAt": "2026-09-18T07:57:51.829Z",
        "__v": 0,
        "latestMessage": "6aaceefe5a7dcc4c6e18891a"
      },
      "messageType": "text",
      "attachments": [],
      "isDeleted": false,
      "deletedFor": [],
      "replyTo": {
        "_id": "6aaceefb5a7dcc4c6e188906",
        "sender": {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        },
        "content": "Hi Priya!",
        "messageType": "text",
        "attachments": []
      },
      "isEdited": false,
      "isForwarded": false,
      "starredBy": [],
      "tags": [],
      "isBroadcast": false,
      "readBy": [],
      "deliveredTo": [],
      "reactions": [],
      "createdAt": "2026-09-18T07:57:48.925Z",
      "updatedAt": "2026-09-18T07:57:48.925Z",
      "__v": 0
    },
    "…1 more"
  ],
  "matchedIds": [
    "6aaceefc5a7dcc4c6e18890f"
  ],
  "matchedCount": 1,
  "chatDetails": {
    "_id": "6aaceef55a7dcc4c6e1888e2",
    "chatName": "sender",
    "isGroupChat": false,
    "users": [
      {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": "",
        "lastSeen": "2026-09-18T07:57:15.923Z"
      },
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "profilePicture": "",
        "lastSeen": "2026-09-18T07:57:17.878Z"
      }
    ],
    "groupAdmin": [],
```

```
    "groupIcon": "",
    "groupDescription": "",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:41.363Z",
    "updatedAt": "2026-09-18T07:57:51.829Z",
    "__v": 0,
    "latestMessage": "6aaceefe5a7dcc4c6e18891a"
  }
}
```


Example

```
GET /api/messages/6aaceef55a7dcc4c6e1888e2?page=1&limit=50
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "isSearchResult": false,
  "chatDetails": {
    "_id": "6aaceef55a7dcc4c6e1888e2",
    "chatName": "sender",
    "isGroupChat": false,
    "users": [
      {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": "",
        "lastSeen": "2026-09-18T07:57:15.923Z"
      },
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "profilePicture": "",
        "lastSeen": "2026-09-18T07:57:17.878Z"
      }
    ],
    "groupAdmin": [],
    "groupIcon": "",
    "groupDescription": "",
    "deletedBy": [],
    "pinnedBy": [],
    "archivedBy": [],
    "admins": [],
    "isLocked": false,
    "mutedBy": [],
    "createdAt": "2026-09-18T07:57:41.363Z",
    "updatedAt": "2026-09-18T07:57:51.829Z",
    "__v": 0,
    "latestMessage": "6aaceefe5a7dcc4c6e18891a"
  },
  "messages": [
    {
      "_id": "6aaceefe5a7dcc4c6e18891a",
      "sender": {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      },
      "content": "",
      "chat": {
        "_id": "6aaceef55a7dcc4c6e1888e2",
        "chatName": "sender",
        "isGroupChat": false,
        "users": [
          "6aaceedb5a7dcc4c6e188876",
          "6aaceedd5a7dcc4c6e18887d"
        ],
        "groupAdmin": [],
        "groupIcon": "",
        "groupDescription": "",
        "deletedBy": [],
        "pinnedBy": [],
        "archivedBy": [],
        "admins": [],
        "isLocked": false,
        "mutedBy": [],
        "createdAt": "2026-09-18T07:57:41.363Z",
        "updatedAt": "2026-09-18T07:57:51.829Z",
        "__v": 0,
        "latestMessage": "6aaceefe5a7dcc4c6e18891a"
      },
      "messageType": "image",
      "attachments": [
        {
          "url": "uploads/messages/images/media-1789718248836-13397368.png",
          "filename": "photo.png",
          "fileType": "image/png",
```

```
        "fileSize": 70,
        "_id": "6aaceefe5a7dcc4c6e18891b"
      }
    ],
    "isDeleted": false,
    "deletedFor": [],
    "isEdited": false,
    "isForwarded": false,
    "starredBy": [],
    "tags": [],
    "isBroadcast": false,
    "readBy": [],
    "deliveredTo": [],
    "reactions": [],
    "createdAt": "2026-09-18T07:57:50.943Z",
    "updatedAt": "2026-09-18T07:57:50.943Z",
    "__v": 0
  },
  {
    "_id": "6aaceefc5a7dcc4c6e18890f",
    "sender": {
      "_id": "6aaceedd5a7dcc4c6e18887d",
      "name": "Priya Verma",
      "profilePicture": ""
    },
    "content": "Hello Rahul",
    "chat": {
      "_id": "6aaceef55a7dcc4c6e1888e2",
      "chatName": "sender",
      "isGroupChat": false,
      "users": [
        "6aaceedb5a7dcc4c6e188876",
        "6aaceedd5a7dcc4c6e18887d"
      ],
      "groupAdmin": [],
      "groupIcon": "",
      "groupDescription": "",
      "deletedBy": [],
      "pinnedBy": [],
      "archivedBy": [],
      "admins": [],
      "isLocked": false,
      "mutedBy": [],
      "createdAt": "2026-09-18T07:57:41.363Z",
      "updatedAt": "2026-09-18T07:57:51.829Z",
      "__v": 0,
      "latestMessage": "6aaceefe5a7dcc4c6e18891a"
    },
    "messageType": "text",
    "attachments": [],
    "isDeleted": false,
    "deletedFor": [],
    "replyTo": {
      "_id": "6aaceefb5a7dcc4c6e188906",
      "sender": {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      },
      "content": "Hi Priya!",
      "messageType": "text",
      "attachments": []
    },
    "isEdited": false,
    "isForwarded": false,
    "starredBy": [],
    "tags": [],
    "isBroadcast": false,
    "readBy": [],
    "deliveredTo": [],
    "reactions": [],
    "createdAt": "2026-09-18T07:57:48.925Z",
```

```
      "updatedAt": "2026-09-18T07:57:48.925Z",
      "__v": 0
    },
    "…1 more"
  ],
  "pagination": {
    "currentPage": 1,
    "totalPages": 1,
    "totalMessages": 3,
    "hasMore": false,
    "limit": 50
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
403	You are not part of this chat	Not a member of the chat
404	Chat not found	

Notes

- Messages you deleted for yourself, or from before you cleared/deleted the chat, are excluded.
- Edited messages have `isEdited: true` and `editedAt`.

Real-time: `message-read-update` to senders of newly read messages.

`DELETE /api/messages/:messageId` — Delete for everyone (path form)

Access: Token · **Body:** `application/json`

Same as `DELETE /api/messages` for one message. The path id is used when no `messageIds` body is sent.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageId`	ObjectId	yes	

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageIds`	array	no	Optional; overrides the path id.

Example — For everyone (path id, no body)

```
DELETE /api/messages/6aacef155a7dcc4c6e18899b
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "1 message(s) deleted successfully",
  "deletedCount": 1,
  "deletedMessageIds": [
    "6aacef155a7dcc4c6e18899b"
  ],
  "skippedCount": 0,
  "skippedMessageIds": []
}
```

Error responses

STATUS	MESSAGE	WHEN
400	messageIds array is required	
404	No messages found	
403	You can only delete your own messages	
500	Server error	

Real-time: `messages-deleted` to the chat room.

PUT /api/messages/:messageId — Edit my message

Access: Token · **Body:** `application/json`

Replaces the text of your own message and marks it `isEdited: true` with `editedAt`. Deleted messages cannot be edited. Everyone in the chat room gets `message-edited`.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageId`	ObjectId	yes	

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`content`	string	yes	New text.

Example

```
PUT /api/messages/6aaceefb5a7dcc4c6e188906
Authorization: Bearer <token>
Content-Type: application/json

{
  "content": "Hi Priya! (edited)"
}
```

Response `200`

```
{
  "success": true,
  "message": {
    "_id": "6aaceefb5a7dcc4c6e188906",
    "sender": {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "profilePicture": ""
    },
    "content": "Hi Priya! (edited)",
    "chat": "6aaceef55a7dcc4c6e1888e2",
    "messageType": "text",
    "attachments": [],
    "isDeleted": false,
    "deletedFor": [],
    "isEdited": true,
    "isForwarded": false,
    "starredBy": [],
    "tags": [],
    "isBroadcast": false,
    "readBy": [
      {
        "user": "6aaceedd5a7dcc4c6e18887d",
        "readAt": "2026-09-18T07:57:54.953Z",
        "_id": "6aacef025a7dcc4c6e188934"
      }
    ],
    "deliveredTo": [
      {
        "user": "6aaceedd5a7dcc4c6e18887d",
        "deliveredAt": "2026-09-18T07:57:54.953Z",
        "_id": "6aacef035a7dcc4c6e188936"
      }
    ],
    "reactions": [],
    "createdAt": "2026-09-18T07:57:47.295Z",
    "updatedAt": "2026-09-18T07:58:01.737Z",
    "__v": 0,
    "editedAt": "2026-09-18T07:58:01.736Z"
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
403	You can only edit your own messages	Editing someone else's message

Notes

- The socket event `edit-message` does the same and is saved too — use either. On failure over the socket you receive `edit-message-error`.

Real-time: `message-edited` to the chat room.

`DELETE /api/messages/:messageId/delete-for-me` — Delete for me (one)

Access: Token · **Body:** —

Hides one message for you only.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageId`	ObjectId	yes	

Example

```
DELETE /api/messages/6aacef105a7dcc4c6e188980/delete-for-me
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Message deleted for you"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Message not found	
500	Server error	

PUT /api/messages/:messageId/read — Mark one message read

Access: Token · **Body:** `application/json`

Marks one message as read and delivered by you and notifies its sender.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageId`	ObjectId	yes	

Example

```
PUT /api/messages/6aacef005a7dcc4c6e188923/read
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Message marked as read"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Message not found	
500	Server error	

Real-time: `message-read-update` to the sender.

PUT /api/messages/:messageId/star — Star / unstar

Access: Token · **Body:** `application/json`

Toggles the star; `isStarred` tells you the new state.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageId`	ObjectId	yes	

Example

```
PUT /api/messages/6aaceefc5a7dcc4c6e18890f/star
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "isStarred": true,
  "message": "Message starred"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Message not found	
500	Server error	

PUT /api/messages/:messageId/react — Add or change reaction

Access: Token · **Body:** `application/json`

Sets your reaction (one per user; a new emoji replaces yours). Returns all reactions.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageId`	ObjectId	yes	

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`emoji`	string	yes	Any emoji.

Example

```
PUT /api/messages/6aaceefc5a7dcc4c6e18890f/react
Authorization: Bearer <token>
Content-Type: application/json

{
  "emoji": "❤️"
}
```

Response `200`

```
{
  "success": true,
  "message": "Reaction added",
  "reactions": [
    {
      "user": "6aaceedb5a7dcc4c6e188876",
      "emoji": "❤️",
      "_id": "6aacef085a7dcc4c6e18894f",
      "reactedAt": "2026-09-18T07:58:00.086Z"
    }
  ]
}
```

Error responses

STATUS	MESSAGE	WHEN
400	Emoji is required	
404	Message not found	
500	Server error	

Notes

- Then emit socket `message-reaction` so others in the chat update live.

`DELETE /api/messages/:messageId/react` — Remove my reaction

Access: Token · **Body:** —

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageId`	ObjectId	yes	

Example

```
DELETE /api/messages/6aaceefc5a7dcc4c6e18890f/react
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Reaction removed"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Message not found	
500	Server error	

PUT /api/messages/:messageId/tag — Add tag

Access: Token · **Body:** `application/json`

Adds a free-text tag to the message (shared, visible to everyone).

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageId`	ObjectId	yes	

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`tag`	string	yes	

Example

```
PUT /api/messages/6aaceefc5a7dcc4c6e18890f/tag
Authorization: Bearer <token>
Content-Type: application/json

{
  "tag": "important"
}
```

Response `200`

```
{
  "success": true,
  "message": "Tag added",
  "tags": [
    "important"
  ]
}
```

Error responses

STATUS	MESSAGE	WHEN
400	Tag is required	
404	Message not found	
500	Server error	

6.5 Status

24-hour stories. Expired statuses are deleted automatically by the database.

`POST /api/status` — Post a status

Access: Token · **Body:** `multipart/form-data` (JSON also works when no file is sent)

Creates a status that expires after 24 hours.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`content`	string	for text	Text, or caption for media.
`mediaType`	string	no	`text` (default) | `image` | `video`.
`backgroundColor`	string	no	Hex, default `#075e54`.
`font`	string	no	Default `Arial`.

Files

FIELD	TYPE	REQUIRED	DESCRIPTION
`media`	image or video	for image/video	Multipart field name.

Example — Text status

```
POST /api/status
Authorization: Bearer <token>
Content-Type: application/json

{
  "content": "Good morning ☀️",
  "mediaType": "text",
  "backgroundColor": "#075e54",
  "font": "Arial"
}
```

Response `201`


```
{
  "success": true,
  "status": {
    "_id": "6aacef295a7dcc4c6e188a07",
    "user": {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "profilePicture": ""
    },
    "content": "Good morning ☀️",
    "mediaType": "text",
    "backgroundColor": "#075e54",
    "font": "Arial",
    "viewedBy": [],
    "expiresAt": "2026-09-19T07:58:33.475Z",
    "createdAt": "2026-09-18T07:58:33.476Z",
    "updatedAt": "2026-09-18T07:58:33.476Z",
    "__v": 0
  }
}
```


Notes

- Then emit socket `new-status` so others refresh.

`GET /api/status` — All statuses

Access: Token + complete profile · **Body:** —

Active statuses grouped by user: `[{ user, statuses: [...] }]`, newest first.

Example

```
GET /api/status
Authorization: Bearer <token>
```

Response `200`


```
{
  "success": true,
  "statuses": [
    {
      "user": {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      },
      "statuses": [
        {
          "_id": "6aacef295a7dcc4c6e188a07",
          "user": {
            "_id": "6aaceedb5a7dcc4c6e188876",
            "name": "Rahul Sharma",
            "profilePicture": ""
          },
          "content": "Good morning ☀️",
          "mediaType": "text",
          "backgroundColor": "#075e54",
          "font": "Arial",
          "viewedBy": [],
          "expiresAt": "2026-09-19T07:58:33.475Z",
          "createdAt": "2026-09-18T07:58:33.476Z",
          "updatedAt": "2026-09-18T07:58:33.476Z",
          "__v": 0
        }
      ]
    }
  ]
}
```


Notes

- With `PRIVACY_ENFORCE=true`: only your own, those with status privacy `everyone`, and those with `contacts` where you are in the poster's contacts. Otherwise every active status on the app is returned.

`GET /api/status/my` — My statuses

Access: Token + complete profile · **Body:** —

Your active statuses with who viewed them.

Example

```
GET /api/status/my
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "statuses": [
    {
      "_id": "6aacef295a7dcc4c6e188a07",
      "user": "6aaceedb5a7dcc4c6e188876",
      "content": "Good morning ☀️",
      "mediaType": "text",
      "backgroundColor": "#075e54",
      "font": "Arial",
      "viewedBy": [],
      "expiresAt": "2026-09-19T07:58:33.475Z",
      "createdAt": "2026-09-18T07:58:33.476Z",
      "updatedAt": "2026-09-18T07:58:33.476Z",
      "__v": 0
    }
  ]
}
```

`PUT /api/status/:statusId/view` — Mark status viewed

Access: Token · **Body:** `application/json`

Adds you to `viewedBy` once. With `PRIVACY_ENFORCE=true`, viewing a status you are not allowed to see returns 403.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`statusId`	ObjectId	yes	

Example

```
PUT /api/status/6aacef295a7dcc4c6e188a07/view
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Status viewed"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Status not found	
403	You cannot view this status	
500	Server error	

`DELETE /api/status/:statusId` — Delete my status

Access: Token · **Body:** —

Deletes your status and its media.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`statusId`	ObjectId	yes	

Example

```
DELETE /api/status/6aacef295a7dcc4c6e188a07
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Status deleted successfully"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Status not found	
403	You can only delete your own status	
500	Server error	

6.6 Channels

One-to-many publishing: admins post, subscribers read.

`POST /api/channels` — Create channel

Access: Token + complete profile · **Body:** `multipart/form-data` (JSON also works when no file is sent)

You become creator and admin, and are auto-subscribed.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelName`	string	yes	
`channelDescription`	string	no	
`category`	string	no	`news` | `entertainment` | `sports` | `technology` | `education` | `other` (default).
`isPrivate`	boolean	no	Default `false`.

Files

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelIcon`	image	no	

Example

```
POST /api/channels
Authorization: Bearer <token>
Content-Type: application/json

{
  "channelName": "Indore Tech News",
  "channelDescription": "Daily tech updates",
  "category": "technology",
  "isPrivate": false
}
```

Response `201`

```
{
  "success": true,
  "channel": {
    "_id": "6aacef2b5a7dcc4c6e188a11",
    "channelName": "Indore Tech News",
    "channelDescription": "Daily tech updates",
    "channelIcon": "",
    "creator": {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "profilePicture": ""
    },
    "admins": [
      {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      }
    ],
    "subscribers": [
      "6aaceedb5a7dcc4c6e188876"
    ],
    "isPrivate": false,
    "inviteLink": "b55c004894cd72026d9ba4acc70b1c3b",
    "category": "technology",
    "createdAt": "2026-09-18T07:58:35.402Z",
    "updatedAt": "2026-09-18T07:58:35.402Z",
    "__v": 0
  }
}
```

Error responses

STATUS	MESSAGE	WHEN
400	Channel name is required	

`GET /api/channels` — Browse channels

Access: Token + complete profile · **Body:** —

Public channels, optionally filtered by category.

Query parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`category`	string	no	
`page`	number	no	
`limit`	number	no	

Example

```
GET /api/channels?category=technology&page=1&limit=20
Authorization: Bearer <token>
```

Response `200`


```
{
  "success": true,
  "channels": [
    {
      "_id": "6aacef2b5a7dcc4c6e188a11",
      "channelName": "Indore Tech News",
      "channelDescription": "Daily tech updates",
      "channelIcon": "",
      "creator": {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      },
      "admins": [
        {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        }
      ],
      "subscribers": [
        "6aaceedb5a7dcc4c6e188876"
      ],
      "isPrivate": false,
      "inviteLink": "b55c004894cd72026d9ba4acc70b1c3b",
      "category": "technology",
      "createdAt": "2026-09-18T07:58:35.402Z",
      "updatedAt": "2026-09-18T07:58:35.402Z",
      "__v": 0,
      "subscriberCount": 1,
      "isSubscribed": false,
      "isAdmin": false,
      "isCreator": false
    }
  ],
  "pagination": {
    "currentPage": 1,
    "totalPages": 1,
    "totalChannels": 1,
    "hasMore": false,
    "limit": 20
  }
}
```

`GET /api/channels/my/owned` — Channels I own

Access: Token + complete profile · **Body:** —

Example

```
GET /api/channels/my/owned
Authorization: Bearer <token>
```

Response `200`


```
{
  "success": true,
  "count": 1,
  "channels": [
    {
      "_id": "6aacef2b5a7dcc4c6e188a11",
      "channelName": "Indore Tech News",
      "channelDescription": "Daily tech updates",
      "channelIcon": "",
      "creator": {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      },
      "admins": [
        {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        }
      ],
      "subscribers": [
        "6aaceedb5a7dcc4c6e188876"
      ],
      "isPrivate": false,
      "inviteLink": "b55c004894cd72026d9ba4acc70b1c3b",
      "category": "technology",
      "createdAt": "2026-09-18T07:58:35.402Z",
      "updatedAt": "2026-09-18T07:58:35.402Z",
      "__v": 0,
      "subscriberCount": 1,
      "isAdmin": true,
      "isCreator": true,
      "unseenCount": 0,
      "totalMessages": 0
    }
  ]
}
```

`GET /api/channels/my/subscribed` — Channels I follow

Access: Token + complete profile · **Body:** —

Example

```
GET /api/channels/my/subscribed
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "count": 1,
  "channels": [
    {
      "_id": "6aacef2b5a7dcc4c6e188a11",
      "channelName": "Indore Tech News",
      "channelDescription": "Daily tech + startup updates",
      "channelIcon": "",
      "creator": {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      },
      "admins": [
        {
          "_id": "6aaceedb5a7dcc4c6e188876",
          "name": "Rahul Sharma",
          "profilePicture": ""
        }
      ],
      "subscribers": [
        "6aaceedb5a7dcc4c6e188876",
        "6aaceedd5a7dcc4c6e18887d"
      ],
      "isPrivate": false,
      "inviteLink": "b55c004894cd72026d9ba4acc70b1c3b",
      "category": "technology",
      "createdAt": "2026-09-18T07:58:35.402Z",
      "updatedAt": "2026-09-18T07:58:39.626Z",
      "__v": 1,
      "subscriberCount": 2,
      "isSubscribed": true,
      "isAdmin": false,
      "isCreator": false,
      "unseenCount": 0,
      "totalMessages": 0
    }
  ]
}
```

`PUT /api/channels/join/:inviteLink` — Join by invite link

Access: Token + complete profile · **Body:** `application/json`

Subscribes you to the channel whose `inviteLink` matches. This is how people join a private channel, which does not appear in `GET /api/channels`.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`inviteLink`	string	yes	The channel's `inviteLink` value (visible to its admins).

Example

```
PUT /api/channels/join/31792a1ab431a3056d05312f898a827b
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Subscribed to channel successfully",
  "subscriberCount": 2,
  "channel": {
    "_id": "6aacef345a7dcc4c6e188a42",
    "channelName": "Core Team",
    "channelDescription": "",
    "channelIcon": "",
    "category": "other",
    "isPrivate": true
  }
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Invalid or expired invite link	Invalid link
500	Server error	

`GET /api/channels/:channelId` — Channel details

Access: Token + complete profile · **Body:** —

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelId`	ObjectId	yes	

Example

```
GET /api/channels/6aacef2b5a7dcc4c6e188a11
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "channel": {
    "_id": "6aacef2b5a7dcc4c6e188a11",
    "channelName": "Indore Tech News",
    "channelDescription": "Daily tech updates",
    "channelIcon": "",
    "creator": {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "profilePicture": ""
    },
    "admins": [
      {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      }
    ],
    "subscribers": [
      "6aaceedb5a7dcc4c6e188876"
    ],
    "isPrivate": false,
    "inviteLink": "b55c004894cd72026d9ba4acc70b1c3b",
    "category": "technology",
    "createdAt": "2026-09-18T07:58:35.402Z",
    "updatedAt": "2026-09-18T07:58:35.402Z",
    "__v": 0,
    "subscriberCount": 1,
    "isSubscribed": false,
    "isAdmin": false,
    "isCreator": false,
    "unseenCount": 0,
    "totalMessages": 0
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
404	Channel not found	

`PUT /api/channels/:channelId/update` — Update channel

Access: Token + complete profile · **Body:** `multipart/form-data` (JSON also works when no file is sent)

Admins only. Replacing the icon deletes the old file.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelId`	ObjectId	yes	

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelName`	string	no	
`channelDescription`	string	no	
`category`	string	no	

Files

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelIcon`	image	no	

Example

```
PUT /api/channels/6aacef2b5a7dcc4c6e188a11/update
Authorization: Bearer <token>
Content-Type: application/json

{
  "channelDescription": "Daily tech + startup updates"
}
```

Response `200`

```
{
  "success": true,
  "channel": {
    "_id": "6aacef2b5a7dcc4c6e188a11",
    "channelName": "Indore Tech News",
    "channelDescription": "Daily tech + startup updates",
    "channelIcon": "",
    "creator": {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "profilePicture": ""
    },
    "admins": [
      {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      }
    ],
    "subscribers": [
      "6aaceedb5a7dcc4c6e188876"
    ],
    "isPrivate": false,
    "inviteLink": "b55c004894cd72026d9ba4acc70b1c3b",
    "category": "technology",
    "createdAt": "2026-09-18T07:58:35.402Z",
    "updatedAt": "2026-09-18T07:58:38.866Z",
    "__v": 0
  }
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Channel not found	
403	Only channel admins can update channel details	

`DELETE /api/channels/:channelId` — Delete channel

Access: Token + complete profile · **Body:** —

Creator only. Deletes all posts and their files.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelId`	ObjectId	yes	

Example

```
DELETE /api/channels/6aacef2b5a7dcc4c6e188a11
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Channel deleted successfully"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Channel not found	
403	Only channel creator can delete channel	
500	Server error	

PUT /api/channels/:channelId/subscribe — Subscribe

Access: Token + complete profile · **Body:** `application/json`

Returns the new `subscriberCount`.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelId`	ObjectId	yes	

Example

```
PUT /api/channels/6aacef2b5a7dcc4c6e188a11/subscribe
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Subscribed to channel successfully",
  "subscriberCount": 2
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Channel not found	

STATUS	MESSAGE	WHEN
500	Server error	

`PUT /api/channels/:channelId/unsubscribe` — Unsubscribe

Access: Token + complete profile · **Body:** `application/json`

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelId`	ObjectId	yes	

Example

```
PUT /api/channels/6aacef2b5a7dcc4c6e188a11/unsubscribe
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Unsubscribed from channel successfully",
  "subscriberCount": 1
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Channel not found	
500	Server error	

`GET /api/channels/:channelId/subscribers` — Subscribers

Access: Token + complete profile · **Body:** —

Admins only.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelId`	ObjectId	yes	

Example

```
GET /api/channels/6aacef2b5a7dcc4c6e188a11/subscribers
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "count": 2,
  "subscribers": [
    {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "mobile": "9000000001",
      "profilePicture": ""
    },
    {
      "_id": "6aaceedd5a7dcc4c6e18887d",
      "name": "Priya Verma",
      "mobile": "9000000002",
      "profilePicture": ""
    }
  ]
}
```


Error responses

STATUS	MESSAGE	WHEN
404	Channel not found	
403	Only admins can view subscriber list	

`POST /api/channels/:channelId/messages` — Post to channel

Access: Token + complete profile · **Body:** `multipart/form-data` (JSON also works when no file is sent)

Admins only. Delivered live to the channel room, and by push to every subscriber (except you) who has channel notifications on.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelId`	ObjectId	yes	

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`content`	string	no	
`messageType`	string	no	`text` | `image` | `video` | `audio` | `document`.

Files

FIELD	TYPE	REQUIRED	DESCRIPTION
`files`	up to 5 files	no	Multipart field name `files`.

Example

```
POST /api/channels/6aacef2b5a7dcc4c6e188a11/messages
Authorization: Bearer <token>
Content-Type: application/json

{
  "content": "New iPhone launch today",
  "messageType": "text"
}
```

Response `201`


```
{
  "success": true,
  "message": {
    "_id": "6aacef315a7dcc4c6e188a32",
    "channel": "6aacef2b5a7dcc4c6e188a11",
    "sender": {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "profilePicture": ""
    },
    "content": "New iPhone launch today",
    "messageType": "text",
    "attachments": [],
    "views": 0,
    "viewedBy": [
      "6aaceedb5a7dcc4c6e188876"
    ],
    "createdAt": "2026-09-18T07:58:41.441Z",
    "updatedAt": "2026-09-18T07:58:41.441Z",
    "__v": 0
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
403	Only channel admins can send messages	Non-admin
404	Channel not found	
500	Server error	

Real-time: `channel-message-received` to the channel room.

`GET /api/channels/:channelId/messages` — Channel posts

Access: Token + complete profile · **Body:** —

Paginated posts; viewing them counts views.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelId`	ObjectId	yes	

Query parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`page`	number	no	

FIELD	TYPE	REQUIRED	DESCRIPTION
`limit`	number	no	

Example

```
GET /api/channels/6aacef2b5a7dcc4c6e188a11/messages?page=1&limit=20
Authorization: Bearer <token>
```

Response `200`


```
{
  "success": true,
  "messages": [
    {
      "_id": "6aacef315a7dcc4c6e188a32",
      "channel": "6aacef2b5a7dcc4c6e188a11",
      "sender": {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      },
      "content": "New iPhone launch today",
      "messageType": "text",
      "attachments": [],
      "views": 0,
      "viewedBy": [
        "6aaceedb5a7dcc4c6e188876"
      ],
      "createdAt": "2026-09-18T07:58:41.441Z",
      "updatedAt": "2026-09-18T07:58:41.441Z",
      "__v": 0
    }
  ],
  "pagination": {
    "currentPage": 1,
    "totalPages": 1,
    "totalMessages": 1,
    "hasMore": false,
    "limit": 20
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
404	Channel not found	
403	Subscribe to view channel messages	

`GET /api/channels/:channelId/unseen-count` — Unseen posts

Access: Token + complete profile · **Body:** —

Number of posts you have not viewed.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelId`	ObjectId	yes	

Example

```
GET /api/channels/6aacef2b5a7dcc4c6e188a11/unseen-count
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "channelId": "6aacef2b5a7dcc4c6e188a11",
  "unseenCount": 0
}
```

`DELETE /api/channels/:channelId/messages/:messageId` — Delete post

Access: Token + complete profile · **Body:** —

Admins only. Deletes the post and its files.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelId`	ObjectId	yes	
`messageId`	ObjectId	yes	

Example

```
DELETE /api/channels/6aacef2b5a7dcc4c6e188a11/messages/6aacef315a7dcc4c6e188a32
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Channel message deleted successfully"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Channel not found	
403	Only channel admins can delete messages	
404	Message not found	
400	Message does not belong to this channel	
500	Server error	

Real-time: `channel-message-deleted` to the channel room.

6.7 Broadcasts

Broadcast lists: one message sent individually to many contacts. Each recipient receives it as a normal 1-to-1 message.

`POST /api/broadcasts` — Create broadcast list

Access: Token + complete profile · **Body:** `multipart/form-data` (JSON also works when no file is sent)

A named list of recipients you can message together.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`broadcastName`	string	yes	Max 50 characters.
`recipients`	array or JSON string	yes	With FormData send `JSON.stringify([...ids])`; with JSON a real array also works.
`description`	string	no	Max 200 characters.

Files

FIELD	TYPE	REQUIRED	DESCRIPTION
`broadcastIcon`	image	no	

Example — recipients is a STRING

```
POST /api/broadcasts
Authorization: Bearer <token>
Content-Type: application/json

{
  "broadcastName": "Family",
  "description": "Family updates",
  "recipients": "[\"6aaceedd5a7dcc4c6e18887d\",\"6aaceedf5a7dcc4c6e188884\"]"
}
```

Response `201`

```
{
  "success": true,
  "broadcast": {
    "_id": "6aacef375a7dcc4c6e188a4f",
    "broadcastName": "Family",
    "creator": {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "profilePicture": ""
    },
    "recipients": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "profilePicture": ""
      },
      {
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "mobile": "9000000003",
        "profilePicture": ""
      }
    ],
    "broadcastIcon": "",
    "description": "Family updates",
    "totalMessagesSent": 0,
    "isMuted": false,
    "isArchived": false,
    "createdAt": "2026-09-18T07:58:47.097Z",
    "updatedAt": "2026-09-18T07:58:47.097Z",
    "__v": 0
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	Broadcast name is required	
400	At least one recipient is required	
400	Invalid recipients format. Must be a JSON array.	
400	One or more recipients are invalid	

`GET /api/broadcasts` — My broadcast lists

Access: Token + complete profile · **Body:** —

Query parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`page`	number	no	
`limit`	number	no	

Example

```
GET /api/broadcasts
Authorization: Bearer <token>
```

Response `200`


```
{
  "success": true,
  "broadcasts": [
    {
      "_id": "6aacef375a7dcc4c6e188a4f",
      "broadcastName": "Family",
      "creator": {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      },
      "recipients": [
        {
          "_id": "6aaceedd5a7dcc4c6e18887d",
          "name": "Priya Verma",
          "mobile": "9000000002",
          "profilePicture": ""
        },
        {
          "_id": "6aaceedf5a7dcc4c6e188884",
          "name": "Amit Kumar",
          "mobile": "9000000003",
          "profilePicture": ""
        }
      ],
      "broadcastIcon": "",
      "description": "Family updates",
      "totalMessagesSent": 0,
      "isMuted": false,
      "isArchived": false,
      "createdAt": "2026-09-18T07:58:47.097Z",
      "updatedAt": "2026-09-18T07:58:47.097Z",
      "__v": 0
    }
  ],
  "pagination": {
    "currentPage": 1,
    "totalPages": 1,
    "totalBroadcasts": 1,
    "hasMore": false,
    "limit": 20
  }
}
```

`GET /api/broadcasts/:broadcastId` — Broadcast list details

Access: Token + complete profile · **Body:** —

Owner only.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`broadcastId`	ObjectId	yes	

Example

```
GET /api/broadcasts/6aacef375a7dcc4c6e188a4f
Authorization: Bearer <token>
```

Response `200`


```
{
  "success": true,
  "broadcast": {
    "_id": "6aacef375a7dcc4c6e188a4f",
    "broadcastName": "Family",
    "creator": {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "profilePicture": ""
    },
    "recipients": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "profilePicture": ""
      },
      {
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "mobile": "9000000003",
        "profilePicture": ""
      }
    ],
    "broadcastIcon": "",
    "description": "Family updates",
    "totalMessagesSent": 0,
    "isMuted": false,
    "isArchived": false,
    "createdAt": "2026-09-18T07:58:47.097Z",
    "updatedAt": "2026-09-18T07:58:47.097Z",
    "__v": 0
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
404	Broadcast list not found	
403	You can only view your own broadcast lists	

`PUT /api/broadcasts/:broadcastId` — Update broadcast list

Access: Token + complete profile · **Body:** `multipart/form-data` (JSON also works when no file is sent)

Owner only. `recipients`, if sent, replaces the whole list.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`broadcastId`	ObjectId	yes	

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`broadcastName`	string	no	
`description`	string	no	

FIELD	TYPE	REQUIRED	DESCRIPTION
`recipients`	array or JSON string	no	

Files

FIELD	TYPE	REQUIRED	DESCRIPTION
`broadcastIcon`	image	no	

Example

```
PUT /api/broadcasts/6aacef375a7dcc4c6e188a4f
Authorization: Bearer <token>
Content-Type: application/json

{
  "broadcastName": "Family ❤️"
}
```

Response `200`


```
{
  "success": true,
  "broadcast": {
    "_id": "6aacef375a7dcc4c6e188a4f",
    "broadcastName": "Family ❤️",
    "creator": {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "profilePicture": ""
    },
    "recipients": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "profilePicture": ""
      },
      {
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "mobile": "9000000003",
        "profilePicture": ""
      }
    ],
    "broadcastIcon": "",
    "description": "Family updates",
    "totalMessagesSent": 0,
    "isMuted": false,
    "isArchived": false,
    "createdAt": "2026-09-18T07:58:47.097Z",
    "updatedAt": "2026-09-18T07:58:49.144Z",
    "__v": 0
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
404	Broadcast list not found	
403	You can only update your own broadcast lists	
400	Invalid recipients format. Must be a JSON array.	
400	At least one recipient is required	

STATUS	MESSAGE	WHEN
400	One or more recipients are invalid	

`DELETE /api/broadcasts/:broadcastId` — Delete broadcast list

Access: Token + complete profile · **Body:** —

Owner only. Messages already delivered stay in recipients' chats.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`broadcastId`	ObjectId	yes	

Example

```
DELETE /api/broadcasts/6aacef375a7dcc4c6e188a4f
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Broadcast list deleted successfully"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Broadcast list not found	
403	You can only delete your own broadcast lists	
500	Server error	

`PUT /api/broadcasts/:broadcastId/add-recipients` — Add recipients

Access: Token + complete profile · **Body:** `application/json`

Owner only.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`broadcastId`	ObjectId	yes	

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`recipients`	array or JSON string	yes	

Example

```
PUT /api/broadcasts/6aacef375a7dcc4c6e188a4f/add-recipients
Authorization: Bearer <token>
Content-Type: application/json

{
  "recipients": "[\"6aaceee15a7dcc4c6e18888b\"]"
}
```

Response `200`


```
{
  "success": true,
  "message": "1 recipient(s) added successfully",
  "broadcast": {
    "_id": "6aacef375a7dcc4c6e188a4f",
    "broadcastName": "Family ❤️",
    "creator": {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "profilePicture": ""
    },
    "recipients": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "profilePicture": ""
      },
      {
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "mobile": "9000000003",
        "profilePicture": ""
      },
      "…1 more"
    ],
    "broadcastIcon": "",
    "description": "Family updates",
    "totalMessagesSent": 0,
    "isMuted": false,
    "isArchived": false,
    "createdAt": "2026-09-18T07:58:47.097Z",
    "updatedAt": "2026-09-18T07:58:50.046Z",
    "__v": 1
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
404	Broadcast list not found	
403	Only broadcast creator can add recipients	
400	Recipients are required	
400	Invalid recipients format. Must be a JSON array.	
400	At least one recipient is required	
400	One or more recipients are invalid	
400	All recipients are already in the broadcast list	
500	Server error	

`PUT /api/broadcasts/:broadcastId/remove-recipients` — Remove recipients

Access: Token + complete profile · **Body:** `application/json`

Owner only.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`broadcastId`	ObjectId	yes	

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`recipients`	array or JSON string	yes	

Example

```
PUT /api/broadcasts/6aacef375a7dcc4c6e188a4f/remove-recipients
Authorization: Bearer <token>
Content-Type: application/json

{
  "recipients": "[\"6aaceee15a7dcc4c6e18888b\"]"
}
```

Response `200`


```
{
  "success": true,
  "message": "1 recipient(s) removed successfully",
  "broadcast": {
    "_id": "6aacef375a7dcc4c6e188a4f",
    "broadcastName": "Family ❤️",
    "creator": {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "profilePicture": ""
    },
    "recipients": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "mobile": "9000000002",
        "profilePicture": ""
      },
      {
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "mobile": "9000000003",
        "profilePicture": ""
      }
    ],
    "broadcastIcon": "",
    "description": "Family updates",
    "totalMessagesSent": 0,
    "isMuted": false,
    "isArchived": false,
    "createdAt": "2026-09-18T07:58:47.097Z",
    "updatedAt": "2026-09-18T07:58:50.946Z",
    "__v": 2
  }
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Broadcast list not found	
403	Only broadcast creator can remove recipients	
400	Recipients are required	
400	Invalid recipients format. Must be a JSON array.	
400	At least one recipient is required	
400	Cannot remove all recipients. Broadcast list must have at least one recipient.	
500	Server error	

`POST /api/broadcasts/:broadcastId/messages` — Send broadcast

Access: Token + complete profile · **Body:** `multipart/form-data` (JSON also works when no file is sent)

Sends the message to each recipient as a separate 1-to-1 message (opening the chat if needed), flagged `isBroadcast`. Each is delivered like a normal message, with push for offline recipients.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`broadcastId`	ObjectId	yes	

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`content`	string	no	
`messageType`	string	no	

Files

FIELD	TYPE	REQUIRED	DESCRIPTION
`files`	up to 5 files	no	Multipart field name `files`.

Example

```
POST /api/broadcasts/6aacef375a7dcc4c6e188a4f/messages
Authorization: Bearer <token>
Content-Type: application/json

{
  "content": "Happy Diwali everyone! 🪔",
  "messageType": "text"
}
```

Response `201`

```
{
  "success": true,
  "message": "Broadcast message sent successfully",
  "broadcastMessage": {
    "_id": "6aacef3c5a7dcc4c6e188a6c",
    "broadcast": "6aacef375a7dcc4c6e188a4f",
    "sender": {
      "_id": "6aaceedb5a7dcc4c6e188876",
      "name": "Rahul Sharma",
      "profilePicture": ""
    },
    "content": "Happy Diwali everyone! 🪔",
    "messageType": "text",
    "attachments": [],
    "recipients": [
      {
        "_id": "6aaceedd5a7dcc4c6e18887d",
        "name": "Priya Verma",
        "profilePicture": ""
      },
      {
        "_id": "6aaceedf5a7dcc4c6e188884",
        "name": "Amit Kumar",
        "profilePicture": ""
      }
    ],
    "deliveryStatus": [
      {
        "user": "6aaceedd5a7dcc4c6e18887d",
        "delivered": false,
        "read": false,
        "_id": "6aacef3c5a7dcc4c6e188a6d"
      },
      {
        "user": "6aaceedf5a7dcc4c6e188884",
        "delivered": false,
        "read": false,
        "_id": "6aacef3c5a7dcc4c6e188a6e"
      }
    ],
    "totalRecipients": 2,
    "totalDelivered": 0,
    "totalRead": 0,
    "isDeleted": false,
    "createdAt": "2026-09-18T07:58:52.313Z",
    "updatedAt": "2026-09-18T07:58:52.313Z",
    "__v": 0
  },
  "recipientsCount": 2
}
```


Error responses

STATUS	MESSAGE	WHEN
404	Broadcast list not found	
403	You can only send messages to your own broadcast lists	
400	No recipients in broadcast list	
500	Server error	

Real-time: `message-received` to each recipient.

`GET /api/broadcasts/:broadcastId/messages` — Sent broadcast messages

Access: Token + complete profile · **Body:** —

History of this list with live `totalDelivered` / `totalRead` per message. Owner only.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`broadcastId`	ObjectId	yes	

Query parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`page`	number	no	
`limit`	number	no	

Example

```
GET /api/broadcasts/6aacef375a7dcc4c6e188a4f/messages
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "messages": [
    {
      "_id": "6aacef3c5a7dcc4c6e188a6c",
      "broadcast": "6aacef375a7dcc4c6e188a4f",
      "sender": {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      },
      "content": "Happy Diwali everyone! 🪔",
      "messageType": "text",
      "attachments": [],
      "recipients": [
        {
          "_id": "6aaceedd5a7dcc4c6e18887d",
          "name": "Priya Verma",
          "profilePicture": ""
        },
        {
          "_id": "6aaceedf5a7dcc4c6e188884",
          "name": "Amit Kumar",
          "profilePicture": ""
        }
      ],
      "deliveryStatus": [
        {
          "user": "6aaceedd5a7dcc4c6e18887d",
          "delivered": false,
          "read": false,
          "_id": "6aacef3c5a7dcc4c6e188a6d"
        },
        {
          "user": "6aaceedf5a7dcc4c6e188884",
          "delivered": false,
          "read": false,
          "_id": "6aacef3c5a7dcc4c6e188a6e"
        }
      ],
      "totalRecipients": 2,
      "totalDelivered": 0,
      "totalRead": 0,
      "isDeleted": false,
      "createdAt": "2026-09-18T07:58:52.313Z",
      "updatedAt": "2026-09-18T07:58:52.313Z",
      "__v": 0
    }
  ],
  "pagination": {
    "currentPage": 1,
    "totalPages": 1,
    "totalMessages": 1,
    "hasMore": false,
    "limit": 50
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
404	Broadcast list not found	
403	You can only view your own broadcast messages	

`GET /api/broadcasts/messages/:messageId/stats` — Broadcast delivery stats

Access: Token + complete profile · **Body:** —

Per-recipient delivered/read status, computed live from each recipient's own message receipts. Owner only.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageId`	ObjectId	yes	A broadcast message id.

Example

```
GET /api/broadcasts/messages/6aacef3c5a7dcc4c6e188a6c/stats
Authorization: Bearer <token>
```

Response `200`


```
{
  "success": true,
  "stats": {
    "totalRecipients": 2,
    "totalDelivered": 0,
    "totalRead": 0,
    "deliveryStatus": [
      {
        "user": {
          "_id": "6aaceedd5a7dcc4c6e18887d",
          "name": "Priya Verma",
          "mobile": "9000000002",
          "profilePicture": ""
        },
        "delivered": false,
        "read": false,
        "_id": "6aacef3c5a7dcc4c6e188a6d"
      },
      {
        "user": {
          "_id": "6aaceedf5a7dcc4c6e188884",
          "name": "Amit Kumar",
          "mobile": "9000000003",
          "profilePicture": ""
        },
        "delivered": false,
        "read": false,
        "_id": "6aacef3c5a7dcc4c6e188a6e"
      }
    ]
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
404	Broadcast message not found	
403	You can only view stats for your own messages	

`DELETE /api/broadcasts/messages/:messageId` — Delete broadcast message

Access: Token · **Body:** —

Removes it from the broadcast history. Owner only.

Path parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageId`	ObjectId	yes	

Example

```
DELETE /api/broadcasts/messages/6aacef3c5a7dcc4c6e188a6c
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Broadcast message deleted successfully"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	Broadcast message not found	
403	You can only delete your own messages	
500	Server error	

6.8 Reports

Report users, messages, groups, channels and statuses for moderation.

`POST /api/reports/user` — Report user

Access: Token · **Body:** `application/json`

Adds a report to the moderation queue with status `pending`.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`userId`	ObjectId	yes	
`reason`	string	yes	See reason values below.
`description`	string	no	Max 500 characters.

Example

```
POST /api/reports/user
Authorization: Bearer <token>
Content-Type: application/json

{
  "userId": "6aaceee15a7dcc4c6e18888b",
  "reason": "harassment",
  "description": "Sending abusive messages"
}
```

Response `201`


```
{
  "success": true,
  "message": "User reported successfully",
  "report": {
    "reporter": "6aaceedd5a7dcc4c6e18887d",
    "reportType": "user",
    "reportedUser": "6aaceee15a7dcc4c6e18888b",
    "reason": "harassment",
    "description": "Sending abusive messages",
    "status": "pending",
    "action": "none",
    "_id": "6aacef425a7dcc4c6e188a9a",
    "createdAt": "2026-09-18T07:58:58.899Z",
    "updatedAt": "2026-09-18T07:58:58.899Z",
    "__v": 0
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	User ID and reason are required	
404	User not found	
500	Server error	

POST /api/reports/message — Report message

Access: Token · **Body:** `application/json`

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`messageId`	ObjectId	yes	
`reason`	string	yes	
`description`	string	no	

Example

```
POST /api/reports/message
Authorization: Bearer <token>
Content-Type: application/json

{
  "messageId": "6aaceefb5a7dcc4c6e188906",
  "reason": "spam"
}
```

Response `201`


```
{
  "success": true,
  "message": "Message reported successfully",
  "report": {
    "reporter": "6aaceedd5a7dcc4c6e18887d",
    "reportType": "message",
    "reportedUser": "6aaceedb5a7dcc4c6e188876",
    "reportedMessage": "6aaceefb5a7dcc4c6e188906",
    "reason": "spam",
    "description": "",
    "status": "pending",
    "action": "none",
    "_id": "6aacef435a7dcc4c6e188a9f",
    "createdAt": "2026-09-18T07:58:59.366Z",
    "updatedAt": "2026-09-18T07:58:59.366Z",
    "__v": 0
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	Message ID and reason are required	
404	Message not found	
500	Server error	

POST /api/reports/group — Report group

Access: Token · **Body:** `application/json`

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`groupId`	ObjectId	yes	A group chat id.
`reason`	string	yes	
`description`	string	no	

Example

```
POST /api/reports/group
Authorization: Bearer <token>
Content-Type: application/json

{
  "groupId": "6aaceef65a7dcc4c6e1888e8",
  "reason": "inappropriate_content"
}
```

Response `201`


```
{
  "success": true,
  "message": "Group reported successfully",
  "report": {
    "reporter": "6aaceedd5a7dcc4c6e18887d",
    "reportType": "group",
    "reportedGroup": "6aaceef65a7dcc4c6e1888e8",
    "reason": "inappropriate_content",
    "description": "",
    "status": "pending",
    "action": "none",
    "_id": "6aacef435a7dcc4c6e188aa2",
    "createdAt": "2026-09-18T07:58:59.823Z",
    "updatedAt": "2026-09-18T07:58:59.823Z",
    "__v": 0
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	Group ID and reason are required	
404	Group not found	
500	Server error	

POST /api/reports/channel — Report channel

Access: Token · **Body:** `application/json`

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`channelId`	ObjectId	yes	
`reason`	string	yes	
`description`	string	no	

Example

```
POST /api/reports/channel
Authorization: Bearer <token>
Content-Type: application/json

{
  "channelId": "6aacef2b5a7dcc4c6e188a11",
  "reason": "fake_news",
  "description": "Wrong info"
}
```

Response `201`

```
{
  "success": true,
  "message": "Channel reported successfully",
  "report": {
    "reporter": "6aaceedd5a7dcc4c6e18887d",
    "reportType": "channel",
    "reportedChannel": "6aacef2b5a7dcc4c6e188a11",
    "reason": "fake_news",
    "description": "Wrong info",
    "status": "pending",
    "action": "none",
    "_id": "6aacef445a7dcc4c6e188aa5",
    "createdAt": "2026-09-18T07:59:00.280Z",
    "updatedAt": "2026-09-18T07:59:00.280Z",
    "__v": 0
  }
}
```

Error responses

STATUS	MESSAGE	WHEN
400	Channel ID and reason are required	
404	Channel not found	
500	Server error	

POST /api/reports/status — Report status

Access: Token · **Body:** `application/json`

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`statusId`	ObjectId	yes	
`reason`	string	yes	
`description`	string	no	

Example

```
POST /api/reports/status
Authorization: Bearer <token>
Content-Type: application/json

{
  "statusId": "6aacef295a7dcc4c6e188a07",
  "reason": "other"
}
```

Response `201`


```
{
  "success": true,
  "message": "Status reported successfully",
  "report": {
    "reporter": "6aaceedd5a7dcc4c6e18887d",
    "reportType": "status",
    "reportedUser": "6aaceedb5a7dcc4c6e188876",
    "reportedStatus": "6aacef295a7dcc4c6e188a07",
    "reason": "other",
    "description": "",
    "status": "pending",
    "action": "none",
    "_id": "6aacef445a7dcc4c6e188aa9",
    "createdAt": "2026-09-18T07:59:00.741Z",
    "updatedAt": "2026-09-18T07:59:00.741Z",
    "__v": 0
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	Status ID and reason are required	
404	Status not found	
500	Server error	

`GET /api/reports/my` — My reports

Access: Token · **Body:** —

Reports you filed, with their moderation status.

Example

```
GET /api/reports/my
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "reports": [
    {
      "_id": "6aacef445a7dcc4c6e188aa9",
      "reporter": "6aaceedd5a7dcc4c6e18887d",
      "reportType": "status",
      "reportedUser": {
        "_id": "6aaceedb5a7dcc4c6e188876",
        "name": "Rahul Sharma",
        "profilePicture": ""
      },
      "reportedStatus": "6aacef295a7dcc4c6e188a07",
      "reason": "other",
      "description": "",
      "status": "pending",
      "action": "none",
      "createdAt": "2026-09-18T07:59:00.741Z",
      "updatedAt": "2026-09-18T07:59:00.741Z",
      "__v": 0
    },
    {
      "_id": "6aacef445a7dcc4c6e188aa5",
      "reporter": "6aaceedd5a7dcc4c6e18887d",
      "reportType": "channel",
      "reportedChannel": {
        "_id": "6aacef2b5a7dcc4c6e188a11",
        "channelName": "Indore Tech News"
      },
      "reason": "fake_news",
      "description": "Wrong info",
      "status": "pending",
      "action": "none",
      "createdAt": "2026-09-18T07:59:00.280Z",
      "updatedAt": "2026-09-18T07:59:00.280Z",
      "__v": 0
    },
    "…3 more"
  ]
}
```

6.9 Contact Cache

Opaque storage for the app's own prepared contact list.

`GET /api/contact-cache` — Get contact cache

Access: Token + complete profile · **Body:** —

Returns the stored string exactly as saved (`"{}"` if none). `JSON.parse` it.

Example

```
GET /api/contact-cache
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "data": "
[{\"id\":\"9000000002\",\"name\":\"Priya\",\"phone\":\"9000000002\",\"avatar\":null,\"isRegistered\":true,\"appUser\":\"6aac
eedd5a7dcc4c6e18887d\"}]",
  "metadata": {
    "userId": "6aaceedb5a7dcc4c6e188876",
    "dataSize": 128,
    "lastUpdated": "2026-09-18T07:59:01.451Z",
    "createdAt": "2026-09-18T07:59:01.452Z",
    "updatedAt": "2026-09-18T07:59:01.452Z"
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
500	Failed to retrieve contact cache	

`PUT /api/contact-cache` — Save contact cache

Access: Token + complete profile · **Body:** `application/json`

Stores a string for you, replacing any previous value entirely (no merge). The server only checks it is valid JSON and within the size limit; it never reads the contents.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`data`	string	yes	**JSON.stringify** of your data. An object or array here is rejected.

Example

```
PUT /api/contact-cache
Authorization: Bearer <token>
Content-Type: application/json

{
  "data": "
[{\"id\":\"9000000002\",\"name\":\"Priya\",\"phone\":\"9000000002\",\"avatar\":null,\"isRegistered\":true,\"appUser\":\"6aac
eedd5a7dcc4c6e18887d\"}]"
}
```

Response `200`


```
{
  "success": true,
  "message": "Contact cache saved successfully",
  "data": {
    "userId": "6aaceedb5a7dcc4c6e188876",
    "dataSize": 128,
    "lastUpdated": "2026-09-18T07:59:01.451Z"
  }
}
```


Error responses

STATUS	MESSAGE	WHEN
400	data must be a string (use JSON.stringify on frontend)	data sent as array — fails
400	data field is required	
400	data must be valid JSON string	
400	Data size ${mb(dataSize)}MB exceeds the ${mb(MAX_DATA_BYTES)}MB limit	
400	Data too large to store. Please reduce the cached contacts payload.	
500	Failed to save contact cache	

Example: data sent as array — fails

Notes

- Limit: 15 MB of `data` (about 120,000 contacts with the app's current fields).

`DELETE /api/contact-cache` — Delete contact cache

Access: Token + complete profile · **Body:** —

Example

```
DELETE /api/contact-cache
Authorization: Bearer <token>
```

Response `200`

```
{
  "success": true,
  "message": "Contact cache deleted successfully"
}
```

Error responses

STATUS	MESSAGE	WHEN
404	No cache found to delete	
500	Failed to delete contact cache	

6.10 Notifications

Device token registration for push notifications.

`POST /api/notifications/save-token` — Register / refresh FCM token

Access: Token · **Body:** `application/json`

Stores this device's current token for the logged-in user, replacing the device's previous one (matched by `deviceId`) and removing it from any other account. Call on **every app start while logged in** and from `onTokenRefresh`.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`token`	string	yes	`messaging().getToken()`.
`deviceType`	string	yes	`android` | `ios` | `web`.
`deviceId`	string	no	Strongly recommended — without it, old tokens of this device are not replaced.

Example

```
POST /api/notifications/save-token
Authorization: Bearer <token>
Content-Type: application/json

{
  "token": "fcm-device-token-rotated",
  "deviceType": "android",
  "deviceId": "a1b2c3d4-install-id"
}
```

Response `200`

```
{
  "success": true,
  "message": "Token saved",
  "totalDevices": 1
}
```

Error responses

STATUS	MESSAGE	WHEN
400	token is required	
400	deviceType must be one of: ${VALID_DEVICE_TYPES.join(", ")}	
404	User not found	

`POST /api/test-notification/send-test-notification` — Send a test push

Access: Token · **Body:** `application/json`

Sends a test notification to the given FCM token. For development.

Body

FIELD	TYPE	REQUIRED	DESCRIPTION
`token`	string	yes	FCM token to send to.

No live example — this endpoint sends a real push through Firebase.

Error responses

STATUS	MESSAGE	WHEN
400	FCM token is required	

Notes

- Not called during verification because it sends a real push through Firebase.

6.11 Media & System

Serving uploaded files, and the health check.

`GET /media/*` — Open an uploaded file

Access: Public · **Body:** —

Every stored file path the API returns (`profilePicture` , `groupIcon` , `attachments[].url` , `mediaUrl` , ...) is a storage key. Request `/media/<key>` and the server redirects (302) to a short-lived signed URL.

Example — Redirects to a signed S3 URL

```
GET /media/uploads/messages/images/media-1789718248836-13397368.png
```

Response `302` → `Location: https://s3.ap-south-1.amazonaws.com/chat.media.storage/uploads/messages/images/media-1789718248836-13397368.png?X-Amz-Algorithm=…&X-Amz-Signature=…`

Notes

- Send your token: `Authorization: Bearer <token>` , or for image components `/media/<key>?token=<token>` . Until the server enables `MEDIA_REQUIRE_AUTH` , requests without a token still work; after that they get 401, and chat attachments are served only to members of that chat (403 otherwise).
- The redirect target expires after ~50 minutes — store the key, not the URL.

`GET /health` — Health check

Access: Public · **Body:** —

Server and Redis status. `clientIp` shows the IP the server sees for you.

Example

```
GET /health
```

Response `200`

```
{
  "success": true,
  "message": "Server is running",
  "redis": "connected",
  "clientIp": "::ffff:127.0.0.1",
  "timestamp": "2026-09-18T07:59:04.376Z"
}
```

---

7. Real-time (Socket.IO)

Socket.IO v4 on the same base URL. One socket per app session, created after login.

Rules

1. **Connect with the JWT in the handshake:** `io(BASE_URL, { auth: { token } })`. The server identifies the user from the token, joins the socket to that user's personal room and marks them online. No `setup` event is needed.
2. **Send messages over HTTP only** (`POST /api/messages`). Never emit `new-message` — it is ignored.
3. **On every `message-received`, emit `message-delivered`.** If that chat is open on screen and the app is active, also emit `message-read`.
4. **Disconnect when the app goes to background; reconnect when it becomes active.** A push is sent only to users with no live socket — a socket left open in background means no notifications.
5. **Emit `join-chat` when a chat screen opens and `leave-chat` when it closes, and re-join after every reconnect.** Typing, deletions, edits and reactions are only sent to sockets inside the chat room.

Connection service (React Native)

```
// services/socket.js
import { io } from 'socket.io-client';
import { AppState } from 'react-native';

const SOCKET_URL = 'https://api.telebookservices.com';
let socket = null;
let appStateSub = null;
let openChatId = null;                      // chat currently on screen

export function connectSocket(token) {
  if (socket) return socket;
  socket = io(SOCKET_URL, {
    transports: ['websocket'],
    auth: { token },                          // rule 1
    reconnection: true,
    reconnectionDelay: 1000,
    reconnectionDelayMax: 10000,
  });

  socket.on('connect', () => {
    // Rooms belong to a socket. After a reconnect, join the open chat again (rule 5).
    if (openChatId) socket.emit('join-chat', openChatId);
  });
  socket.on('connected', () => { /* marked online */ });
  socket.on('connect_error', (err) => console.warn('socket:', err.message));

  appStateSub = AppState.addEventListener('change', (state) => {   // rule 4
    if (state === 'active') { if (!socket.connected) socket.connect(); }
    else if (state === 'background') socket.disconnect();          // not on 'inactive' (iOS)
  });
  return socket;
}

export function disconnectSocket() {       // on logout
  appStateSub?.remove(); socket?.disconnect(); socket = null; openChatId = null;
}
export const getSocket  = () => socket;
export const setOpenChat = (id) => { openChatId = id; };
export const getOpenChat = () => openChatId;
```

Message lifecycle

STEP	WHO	WHAT
1	Sender A	`POST /api/messages` → show the returned message with ✓
2	Server	emits `message-received` to every other member; members without a live socket get a push
3	Receiver B	on `message-received` emits `message-delivered` — even on another screen
4	Server	emits `message-delivered-update` to A → ✓✓
5a	B, chat already open	emits `message-read`
5b	B, opens the chat later	calls `GET /api/messages/:chatId` — the server marks everything read; nothing to emit
6	Server	emits `message-read-update` to A for each message (5a and 5b) → blue ✓✓

```
socket.on('message-received', (message) => {
  const chatId = message.chat?._id ?? message.chat;
  socket.emit('message-delivered', { messageId: message._id });                  // always
  if (chatId === getOpenChat() && AppState.currentState === 'active') {
    socket.emit('message-read', { messageId: message._id, chatId });             // only if visible
  }
  upsertMessage(message);   // chat screen, chat-list preview, unread badge
});

export async function openChat(chatId) {
  setOpenChat(chatId);
  getSocket()?.emit('join-chat', chatId);
  return api.get(`/api/messages/${chatId}`);   // also marks read and notifies senders
}
export function closeChat(chatId) { getSocket()?.emit('leave-chat', chatId); setOpenChat(null); }
```

Ticks

For messages you sent. Update `deliveredTo` / `readBy` when the update events arrive, then derive:

TICK	RULE
✓ sent	saved; no recipient in `deliveredTo`
✓✓ delivered	every recipient in `deliveredTo`
blue ✓✓ read	every recipient in `readBy`

```
socket.on('message-delivered-update', ({ messageId, userId }) => addReceipt(messageId, 'deliveredTo', userId));
socket.on('message-read-update', ({ messageId, userId }) => {
  addReceipt(messageId, 'readBy', userId);
  addReceipt(messageId, 'deliveredTo', userId);   // read implies delivered
});

export function tickState(message, myId, chatUsers) {
  const others = chatUsers.map((u) => String(u._id)).filter((id) => id !== String(myId));
  const has = (list, id) => (list || []).some((r) => String(r.user) === id);
  if (others.every((id) => has(message.readBy, id)))      return 'read';
  if (others.every((id) => has(message.deliveredTo, id))) return 'delivered';
  return 'sent';
}
```

In groups `every` shows blue once all members have read; use `some` for blue on first read.

Events: app → server

EVENT	PAYLOAD	WHEN
`message-delivered`	`{ messageId }`	on every `message-received`
`message-read`	`{ messageId, chatId }`	message received while its chat is on screen
`join-chat`	`chatId`	chat screen opens; again after reconnect
`leave-chat`	`chatId`	chat screen closes
`typing`	`{ chatId }`	user starts typing
`stop-typing`	`{ chatId }`	stops typing or sends
`get-user-status`	`{ userId }`	opening a chat, to show online / last seen
`message-reaction`	`{ messageId, chatId, emoji }`	after `PUT /api/messages/:id/react`
`delete-message`	`{ messageId, chatId }`	optional relay — `DELETE /api/messages` already notifies
`edit-message`	`{ messageId, chatId, newContent }`	edit your own message — saved, same as `PUT /api/messages/:id`
`user-added-to-group`	`{ chat, newUserId }`	after `PUT /api/chats/group/add`
`user-removed-from-group`	`{ chat, removedUserId }`	after `PUT /api/chats/group/remove`
`group-info-updated`	`{ chat }`	after renaming / changing description
`new-status`	status object	after `POST /api/status`
`join-channel` / `leave-channel`	`channelId`	channel screen opens / closes
`call-user`	`{ to, offer, from, callType }`	start a call (`to` = user id)
`answer-call`	`{ to, answer }`	accept
`ice-candidate`	`{ to, candidate }`	WebRTC ICE
`end-call` / `reject-call`	`{ to }`	hang up / decline
~~`new-message`~~	—	**deprecated, ignored** — remove
~~`heartbeat`~~	—	**deprecated**, no longer needed
`setup`	`{ _id }`	legacy, only without a token; with a token the `_id` is ignored

Events: server → app

Events sent to a *chat room* arrive only after `join-chat`.

EVENT	PAYLOAD	REACHES	MEANING
`connected`	—	this socket	marked online
`message-received`	message (sender populated)	user room	new message, forwarded message, broadcast, group-created system message
`message-delivered-update`	`{ messageId, userId, chatId }`	sender	a recipient received it
`message-read-update`	`{ messageId, userId, chatId }`	sender	a recipient read it
`user-online`	`userId`	everyone*	came online
`user-offline`	`userId`	everyone*	last socket closed
`user-status-result`	`{ userId, status, hidden? }`	this socket	reply to `get-user-status` ; `online` / `offline` . `hidden: true` when their last-seen privacy hides it from you (with `PRIVACY_ENFORCE`)
`typing` / `stop-typing`	`{ chatId, userId }`	chat room	typing indicator
`messages-deleted`	`{ messageIds, chatId }`	chat room	from `DELETE /api/messages`
`message-deleted`	`{ messageId, chatId }`	chat room	from the `delete-message` relay
`message-edited`	`{ messageId, newContent, chatId, editedAt }`	chat room	a message was edited

EVENT	PAYLOAD	REACHES	MEANING
`edit-message-error`	`{ messageId, message }`	this socket	your `edit-message` was refused (not yours, deleted, empty)
`message-reaction-update`	`{ messageId, emoji, userId }`	chat room	reaction added
`added-to-group`	chat	added user	you were added
`removed-from-group`	`chatId`	removed user	you were removed
`group-updated`	chat	members / chat room	members or info changed
`status-update`	status	everyone	new status
`channel-message-received`	channel post	channel room	new post
`channel-message-deleted`	`{ channelId, messageId }`	channel room	post deleted
`incoming-call`	`{ from, offer, callType }`	callee	ring
`call-answered`	answer	caller	accepted
`ice-candidate`	candidate	peer	WebRTC ICE
`call-ended` / `call-rejected`	—	peer / caller	hung up / declined

* With `PRIVACY_ENFORCE`, presence goes only to users allowed to see that person's last seen.

Presence and typing

```
socket.on('user-online',  (userId) => setPresence(userId, 'online'));
socket.on('user-offline', (userId) => setPresence(userId, 'offline', new Date()));
socket.emit('get-user-status', { userId: otherUserId });
socket.on('user-status-result', ({ userId, status }) => setPresence(userId, status));

let typingTimer;
function onChangeText() {
  if (!typingTimer) socket.emit('typing', { chatId });
  clearTimeout(typingTimer);
  typingTimer = setTimeout(() => { socket.emit('stop-typing', { chatId }); typingTimer = null; }, 2000);
}
```

A user is online while any of their devices has a live socket. If the OS kills the app without closing the socket, the server notices after up to ~85 seconds; messages in that window get no push. Normal backgrounding (rule 4) has no such gap.

Common mistakes

- Emitting `new-message` after sending — ignored; the server already delivered it.
- Not re-joining the chat room after reconnect — typing, deletes and reactions stop silently.
- Emitting `message-read` for a chat that is not visible — the sender sees blue ticks for an unread message.
- Keeping the socket connected in background — user looks online and gets no pushes.
- Reading `data.isGroup` from a push as a boolean — it is the string `"true"` / `"false"`.

8. Push notifications (FCM)

Firebase Cloud Messaging. The server decides when to push; the app's job is to keep the server's copy of the device token current, and to handle taps.

When the server sends a push

EVENT	PUSHED TO	SKIPPED WHEN
New chat message (also forwarded, broadcast, "created group")	every other member	the member has a **live socket**; the member **muted** the chat; `messageNotifications` (1-to-1) or `groupNotifications` (group) is `false`; the member **blocked** the sender (1-to-1)
Channel post	every subscriber except the poster	`channelNotifications` is `false`. Sent **even if the subscriber is online**, because channel posts reach the socket only inside the channel room
Status, reaction, edit, receipt, typing	—	never pushed

- `sound: false` in notification settings sends the push silently.
- Tokens Firebase reports as dead are deleted from the user automatically.
- **One device belongs to one account.** Registering a token or `deviceId` removes it from any other account, so switching accounts on a phone never shows the previous account's messages.

What the app must do

1. **Ask permission** (Android 13+ and iOS require it).
2. **Send the token at login** — `fcmToken`, `deviceType`, `deviceId` in `POST /api/auth/verify-otp`.
3. **Send it again on every app start while logged in, and whenever it changes** — `POST /api/notifications/save-token`. This is the rule most often missed: after a reinstall, cleared data or an Android backup restore, the saved session survives but Firebase issues a **new** token and does not fire a refresh event. If the app only sends the token at login, the server keeps the dead one and the user gets no notifications (server log: `registration-token-not-registered`).
4. **On logout** call `POST /api/auth/logout` with the `deviceId`, then `messaging().deleteToken()`, then delete the saved JWT.
5. **Disconnect the socket in background** (socket rule 4). A connected socket makes the user count as online, and online users get no chat pushes.

`deviceId` must be stable for the install: generate a UUID on first launch and keep it in storage (or use `react-native-device-info`'s `getUniqueId()`). It lets the server replace this device's old token instead of piling up duplicates.

Code (React Native Firebase)

```
// services/push.js
import messaging from '@react-native-firebase/messaging';
import { Platform, PermissionsAndroid } from 'react-native';
import { api } from './api';            // axios/fetch wrapper that adds Authorization
import { getDeviceId } from './deviceId';

export async function requestPushPermission() {
  if (Platform.OS === 'android' && Platform.Version >= 33) {
    await PermissionsAndroid.request(PermissionsAndroid.PERMISSIONS.POST_NOTIFICATIONS);
  }
  if (Platform.OS === 'ios') await messaging().requestPermission();
}

/** Token for the login call (rule 2). */
export async function getLoginPushFields() {
  return { fcmToken: await messaging().getToken(), deviceType: Platform.OS, deviceId: await getDeviceId() };
}

/** Call on every app start after the user is logged in (rule 3). */
export async function syncPushToken() {
  const token = await messaging().getToken();
  await api.post('/api/notifications/save-token', { token, deviceType: Platform.OS, deviceId: await getDeviceId() });
}

let unsubscribeRefresh;
export function listenForTokenRefresh() {
  unsubscribeRefresh?.();
  unsubscribeRefresh = messaging().onTokenRefresh(async (token) => {
    await api.post('/api/notifications/save-token', { token, deviceType: Platform.OS, deviceId: await getDeviceId() });
  });
}

/** Rule 4. */
export async function logoutPush() {
  unsubscribeRefresh?.();
  await api.post('/api/auth/logout', { deviceId: await getDeviceId() }).catch(() => {});
  await messaging().deleteToken();
}
```

```
// Tap handling — open the right screen
export function registerNotificationTaps(navigate) {
  const open = (msg) => {
    const d = msg?.data || {};
    if (d.entity === 'chat')    navigate('Chat',    { chatId: d.entityId });
    if (d.entity === 'channel') navigate('Channel', { channelId: d.entityId });
  };
  messaging().getInitialNotification().then(open);   // app was closed
  return messaging().onNotificationOpenedApp(open);   // app was in background
}

// Foreground: chat pushes do not arrive while the socket is connected.
// Channel posts can — show an in-app banner unless that channel is on screen.
messaging().onMessage(async (msg) => {
  if (msg.data?.entity === 'channel' && msg.data.entityId !== currentChannelId()) {
    showInAppBanner(msg.notification?.title, msg.notification?.body);
  }
});
```

```
// index.js — required by React Native Firebase for background delivery
import messaging from '@react-native-firebase/messaging';
messaging().setBackgroundMessageHandler(async () => {});
```

App start order: restore JWT → if logged in: `requestPushPermission()`, `syncPushToken()`, `listenForTokenRefresh()`, `registerNotificationTaps(...)`, connect the socket.

Payload

The push carries a `notification` (shown by the OS in background) and `data` (all values are **strings**).

FIELD	CHAT MESSAGE	CHANNEL POST
`notification.title`	`Rahul Sharma`, or `Weekend Trip • Rahul Sharma` in a group	channel name
`notification.body`	text (first 80 chars) or `📷 Image` · `🎥 Video` · `🎵 Audio` · `📄 Document`	text (first 80 chars), `📷 Image`, `🎥 Video` or `New Post in channel`
`data.entity`	`chat`	`channel`
`data.entityId`	chat id	channel id
`data.senderId`	sender's user id	poster's user id
`data.senderName`	sender's name	poster's name
`data.isGroup`	`"true"` / `"false"` — compare with `=== "true"`	`"true"`

Troubleshooting

SERVER LOG OR SYMPTOM	MEANING	FIX
`⚠️ User <id> ke paas koi valid FCM token nahi`	the server has no token for that user	the app never sent one, or it was deleted as dead — apply rule 3
`❌ Token Failed … messaging/registration-token-not-registered`	the saved token is dead (reinstall, cleared data, `deleteToken()`, uninstall)	the server deletes it automatically; the app must send the current token on next start (rule 3)
`messaging/invalid-registration-token`	not a valid token string	send exactly what `getToken()` returns
`messaging/mismatched-credential`, `sender-id-mismatch`, `third-party-auth-error`	the app was built with a different Firebase project than the server's service account	use the same project's `google-services.json` / `GoogleService-Info.plist`
No push while the app is open	expected — a live socket means the message arrived over the socket	—
No push for a muted chat or with notifications turned off	expected	—
Pushes keep arriving after logout	the app did not call `POST /api/auth/logout`	rule 4

Testing a device

`POST /api/test-notification/send-test-notification` with `{ "token": "<getToken() result>" }` sends a fixed test push straight to that token, bypassing chat logic. If this fails, the problem is the token or the Firebase configuration, not the chat flow. It returns `500` with Firebase's error message when the token is rejected.

9. Data models

Generated from the Mongoose schemas. Every document also has `_id`; models with timestamps have `createdAt` and `updatedAt`. Fields ending in `[]` are array items.

User

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`name`	String		null	max 50 chars
`mobile`	String	yes		unique
`district`	String		null	
`city`	String		null	
`state`	String		null	
`profilePicture`	String			
`about`	String		Hey there! I am using TeleBook	max 139 chars
`isOnline`	Boolean		false	
`lastSeen`	Date		now	
`socketId`	String			
`otp`	String			
`otpExpiry`	Date			
`isVerified`	Boolean		false	
`isDeleted`	Boolean		false	
`deletedAt`	Date			
`savedContacts`	array			
`savedContacts[].user`	ObjectId → User			
`savedContacts[].displayName`	String			
`savedContacts[].addedAt`	Date		now	
`savedContacts[]._id`	ObjectId			
`privacySettings.lastSeen`	String		everyone	one of: `everyone`, `contacts`, `nobody`
`privacySettings.profilePhoto`	String		everyone	one of: `everyone`, `contacts`, `nobody`

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`privacySettings.about`	String		everyone	one of: `everyone`, `contacts`, `nobody`
`privacySettings.status`	String		contacts	one of: `everyone`, `contacts`, `nobody`
`inviteCode`	String			unique
`blockedUsers`	array → User			
`reportedUsers`	array			
`reportedUsers[].user`	ObjectId → User			
`reportedUsers[].reason`	String			
`reportedUsers[].reportedAt`	Date		now	
`reportedUsers[]._id`	ObjectId			
`notificationSettings.messageNotifications`	Boolean		true	
`notificationSettings.groupNotifications`	Boolean		true	
`notificationSettings.statusNotifications`	Boolean		true	
`notificationSettings.channelNotifications`	Boolean		true	
`notificationSettings.sound`	Boolean		true	
`notificationSettings.vibration`	Boolean		true	
`fcmTokens`	array			
`fcmTokens[].token`	String	yes		
`fcmTokens[].deviceType`	String	yes		one of: `android`, `ios`, `web`
`fcmTokens[].deviceId`	String			
`fcmTokens[].createdAt`	Date		now	
`cachedContactsData`	mixed (any JSON)		(computed)	
`_id`	ObjectId			
`createdAt`	Date			
`updatedAt`	Date			

Chat

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`chatName`	String			
`isGroupChat`	Boolean		false	
`users`	array → User			
`latestMessage`	ObjectId → Message			
`groupAdmin`	array → User			
`groupIcon`	String			
`groupDescription`	String			
`deletedBy`	array → User			
`mutedBy`	array			
`mutedBy[].user`	ObjectId → User			
`mutedBy[].mutedUntil`	Date			
`mutedBy[]._id`	ObjectId			
`pinnedBy`	array → User			
`archivedBy`	array → User			
`admins`	array → User			
`isLocked`	Boolean		false	
`_id`	ObjectId			
`createdAt`	Date			
`updatedAt`	Date			

Message

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`sender`	ObjectId → User	yes		
`content`	String			
`chat`	ObjectId → Chat	yes		
`messageType`	String		text	one of: `text`, `image`, `video`, `audio`, `document`, `emoji`

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`attachments`	array			
`attachments[].url`	String			
`attachments[].filename`	String			
`attachments[].fileType`	String			
`attachments[].fileSize`	Number			
`attachments[]._id`	ObjectId			
`readBy`	array			
`readBy[].user`	ObjectId → User			
`readBy[].readAt`	Date		now	
`readBy[]._id`	ObjectId			
`deliveredTo`	array			
`deliveredTo[].user`	ObjectId → User			
`deliveredTo[].deliveredAt`	Date		now	
`deliveredTo[]._id`	ObjectId			
`isDeleted`	Boolean		false	
`deletedFor`	array → User			
`replyTo`	ObjectId → Message			
`isEdited`	Boolean		false	
`editedAt`	Date			
`isForwarded`	Boolean		false	
`forwardedFrom`	ObjectId → User			
`starredBy`	array → User			

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`reactions`	array			
`reactions[].user`	ObjectId → User			
`reactions[].emoji`	String			
`reactions[].reactedAt`	Date		now	
`reactions[]._id`	ObjectId			
`tags`	array			
`isBroadcast`	Boolean		false	
`broadcastId`	ObjectId → Broadcast			
`broadcastMessage`	ObjectId → BroadcastMessage			
`_id`	ObjectId			
`createdAt`	Date			
`updatedAt`	Date			

Status

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`user`	ObjectId → User	yes		
`content`	String			
`mediaType`	String		text	one of: `text`, `image`, `video`
`mediaUrl`	String			
`backgroundColor`	String		#075e54	
`font`	String		Arial	
`viewedBy`	array			
`viewedBy[].user`	ObjectId → User			
`viewedBy[].viewedAt`	Date		now	
`viewedBy[]._id`	ObjectId			
`expiresAt`	Date		now + 24 h	
`_id`	ObjectId			
`createdAt`	Date			
`updatedAt`	Date			

Channel

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`channelName`	String	yes		
`channelDescription`	String			
`channelIcon`	String			
`creator`	ObjectId → User	yes		
`admins`	array → User			
`subscribers`	array → User			
`isPrivate`	Boolean		false	
`inviteLink`	String			unique
`category`	String		other	one of: `news`, `entertainment`, `sports`, `technology`, `education`, `other`
`_id`	ObjectId			
`createdAt`	Date			
`updatedAt`	Date			

ChannelMessage

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`channel`	ObjectId → Channel	yes		
`sender`	ObjectId → User	yes		
`content`	String			
`messageType`	String		text	one of: `text`, `image`, `video`, `audio`, `document`
`attachments`	array			
`attachments[].url`	String			
`attachments[].filename`	String			
`attachments[].fileType`	String			
`attachments[].fileSize`	Number			
`attachments[]._id`	ObjectId			
`views`	Number		0	
`viewedBy`	array → User			
`_id`	ObjectId			
`createdAt`	Date			
`updatedAt`	Date			

Broadcast

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`broadcastName`	String	yes		max 50 chars
`creator`	ObjectId → User	yes		
`recipients`	array → User			
`broadcastIcon`	String			
`description`	String			max 200 chars
`lastMessage`	ObjectId → BroadcastMessage			
`totalMessagesSent`	Number		0	
`isMuted`	Boolean		false	
`isArchived`	Boolean		false	
`_id`	ObjectId			
`createdAt`	Date			
`updatedAt`	Date			

BroadcastMessage

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`broadcast`	ObjectId → Broadcast	yes		
`sender`	ObjectId → User	yes		
`content`	String			
`messageType`	String		text	one of: `text`, `image`, `video`, `audio`, `document`
`attachments`	array			
`attachments[].url`	String			
`attachments[].filename`	String			
`attachments[].fileType`	String			
`attachments[].fileSize`	Number			
`attachments[]._id`	ObjectId			
`recipients`	array → User			
`deliveryStatus`	array			

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`deliveryStatus[].user`	ObjectId → User			
`deliveryStatus[].delivered`	Boolean		false	
`deliveryStatus[].deliveredAt`	Date			
`deliveryStatus[].read`	Boolean		false	
`deliveryStatus[].readAt`	Date			
`deliveryStatus[]._id`	ObjectId			
`totalRecipients`	Number		0	
`totalDelivered`	Number		0	
`totalRead`	Number		0	
`isDeleted`	Boolean		false	
`_id`	ObjectId			
`createdAt`	Date			
`updatedAt`	Date			

Report

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`reporter`	ObjectId → User	yes		
`reportType`	String	yes		one of: `user`, `message`, `group`, `channel`, `status`
`reportedUser`	ObjectId → User			
`reportedMessage`	ObjectId → Message			
`reportedGroup`	ObjectId → Chat			
`reportedChannel`	ObjectId → Channel			
`reportedStatus`	ObjectId → Status			
`reason`	String	yes		one of: `spam`, `harassment`, `inappropriate_content`, `violence`, `hate_speech`, `fake_news`, `scam`, `other`
`description`	String			max 500 chars
`status`	String		pending	one of: `pending`, `reviewed`, `resolved`, `dismissed`
`reviewedBy`	ObjectId → User			
`reviewedAt`	Date			
`action`	String		none	one of: `none`, `warning`, `content_removed`, `user_banned`, `account_suspended`
`_id`	ObjectId			
`createdAt`	Date			
`updatedAt`	Date			

ContactCache

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`userId`	ObjectId → User	yes		unique
`cachedData`	String		{}	

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
`lastUpdated`	Date		(computed)	
`dataSize`	Number		0	
`_id`	ObjectId			
`createdAt`	Date			
`updatedAt`	Date			

User fields **never** returned for other users: `otp`, `otpExpiry`, `fcmTokens`, `cachedContactsData`, `savedContacts`, `reportedUsers`, `socketId`.

10. Enumerations

FIELD	ALLOWED VALUES
`privacySettings.lastSeen`	`everyone`, `contacts`, `nobody`
`privacySettings.profilePhoto`	`everyone`, `contacts`, `nobody`
`privacySettings.about`	`everyone`, `contacts`, `nobody`
`privacySettings.status`	`everyone`, `contacts`, `nobody`
`fcmTokens[].deviceType`	`android`, `ios`, `web`
`messageType`	`text`, `image`, `video`, `audio`, `document`, `emoji`
`mediaType`	`text`, `image`, `video`
`category`	`news`, `entertainment`, `sports`, `technology`, `education`, `other`
`messageType`	`text`, `image`, `video`, `audio`, `document`
`reportType`	`user`, `message`, `group`, `channel`, `status`
`reason`	`spam`, `harassment`, `inappropriate_content`, `violence`, `hate_speech`, `fake_news`, `scam`, `other`
`status`	`pending`, `reviewed`, `resolved`, `dismissed`
`action`	`none`, `warning`, `content_removed`, `user_banned`, `account_suspended`
mute `duration`	hours as a number; `0` = forever
`deviceType`	`android`, `ios`, `web`

11. Server configuration

Environment variables that change API behaviour. The three `*_REQUIRE_AUTH` / `PRIVACY_ENFORCE` flags are off so the current app keeps working; turn each on once the app is updated as described.

VARIABLE	DEFAULT	EFFECT	TURN ON / CHANGE WHEN
`PRIVACY_ENFORCE`	`false`	Applies privacy settings everywhere (profiles, lists, messages, status feed, presence).	After the app shows `lastSeen: null` as "hidden" and handles empty `profilePicture` / `about`.
`MEDIA_REQUIRE_AUTH`	`false`	Requires a token on `/media/*`; chat attachments only for chat members.	After the app appends `?token=` (or sends the header) on every media URL.
`SOCKET_REQUIRE_AUTH`	`false`	Rejects socket connections without a valid JWT in the handshake.	After the app connects with `io(url, { auth: { token } })`.
`APP_BASE_URL`	`https://yourapp.com`	Base of `inviteLink` returned by `GET /api/users/invite-code`.	Set to the real app / web domain.
`CONTACT_CACHE_MAX_MB`	`15`	Size limit for `PUT /api/contact-cache` `data`.	Leave as is; MongoDB caps a document at 16 MB.
`TRUST_PROXY_HOPS`	`2`	Proxies in front of the server (load balancer + nginx), so rate limiting sees real client IPs.	Set to 1 if `GET /health` shows a proxy IP in `clientIp`.
`KEEP_ALIVE_TIMEOUT_MS`	`65000`	HTTP keep-alive; must exceed the load balancer idle timeout.	Leave as is.
`RATE_LIMIT_MAX_REQUESTS`	`100`	Requests per minute per IP.	

12. Known limitations

Current server-side limitations. These are not app bugs; work around them as described.

1. **Three protections are switched off by default.** `PRIVACY_ENFORCE`, `MEDIA_REQUIRE_AUTH` and `SOCKET_REQUIRE_AUTH` keep the current app working. Until they are on, privacy settings are not applied, media URLs work without a token, and sockets can connect with only `setup`. See Server configuration for when to enable each.
2. **Invite link domain.** Until `APP_BASE_URL` is set, `inviteLink` is `https://yourapp.com/join/<code>`.
3. **Channel posts push to online subscribers too.** A channel post is sent over the socket only to users inside the channel room, so subscribers who are online elsewhere in the app still get a push. This is intentional.
4. **Group settings.** Any member can change group info and add members (the WhatsApp default). There is no "only admins can edit" setting yet.
5. **`GET /api/status` is not paginated.** All visible active statuses are returned in one response.
6. **Public stickers.** `GET /api/users/stickers` needs no token; stickers are static built-in images.

13. Verification checklist

Run on two devices with users A and B. Every item should pass.

- [ ] New number: send-otp → verify-otp returns `isNewUser: true` → complete-profile → chat list loads.
- [ ] Existing number: verify-otp returns `isNewUser: false` and goes straight to the chat list.
- [ ] Killing the app and reopening restores the session via `GET /api/auth/me`.
- [ ] A new user who skips profile completion is routed to it on the first 403.
- [ ] A and B both online, B on the chat list: A sends a message → B's list updates live with an unread badge, B gets no push, A sees ✓✓.
- [ ] B opens the chat → A's ticks turn blue within about a second; B's badge clears.
- [ ] B sends the app to background → A sends → B gets a push; tapping it opens that chat.
- [ ] Image message: upload, send, and display through `/media/<key>` on both sides.
- [ ] Reply, forward to another chat, star, react, delete for me, delete for everyone — each updates both devices.
- [ ] Search inside a chat highlights `matchedIds`.
- [ ] Group: create (members get it live), rename, add and remove a member, lock (non-admin blocked with 403), unlock, leave, delete.
- [ ] Pin moves a chat to the top; mute and archive flags show correctly.
- [ ] Status: post text status, B sees and views it, A sees B in viewers, delete.
- [ ] Channel: create, B subscribes, A posts, B receives it live, unseen count, delete post.
- [ ] Broadcast: create with B and C, send — B and C each receive it as a 1-to-1 message.
- [ ] Edit a sent message — both devices show the new text with an "edited" label, and it survives a reload.
- [ ] B blocks A: A's new messages stay on one tick and never reach B; B cannot send to A (403). Unblock restores delivery.
- [ ] B mutes the chat and backgrounds the app: A's messages raise B's unread badge but send no push.
- [ ] Delete for me on the last message: the chat list preview shows the previous message.
- [ ] A deletes their account: B's chat with A still opens, shows "Deleted account", and B cannot send to it.
- [ ] Private channel: not listed in browse; joining with its invite link works.
- [ ] Broadcast: after B opens the chat, the stats show B as delivered and read.
- [ ] Report a message.
- [ ] Reinstall the app on B while staying logged in (or clear its token): after reopening, A's message still reaches B as a push.
- [ ] Log out on B, then send from A: B's phone gets no push. Log in as another account on B's phone: it never shows the first account's notifications.
- [ ] Airplane mode for 20 s then off: socket reconnects and the open chat still receives typing and reactions.